Master of Science in Artificial Intelligence, 2026-2028



Knowledge Representation and Reasoning

National University of Science and Technology
POLITEHNICA Bucharest

Department of Computer Science

Fall 2026

Adina Magda Florea

Lecture 4

Bayesian Networks

- Uncertain reasoning
- Bayesian inference
- Bayesian Networks
- Inference algorithms in BN
  - Exact inference methods
  - Approximate inference methods

1. Uncertain knowledge

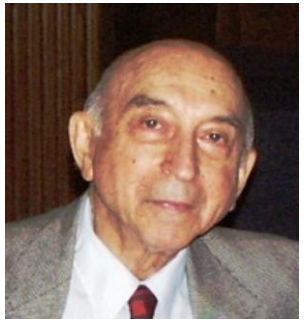
Different models in AI to represent uncertain knowledge

- Probabilistic models
  - Probability theory
  - **Probabilistic graphical models**
    - **Baysian networks**
    - **Markov models**
- Fuzzy logic
  - and others
- Heuristic models (Certainty Factors - MYCIN)



Thomas Bayes

1701 – 1761 (doubtful whether the portrait is actually of him)



Lotfi A. Zadeh

1921 - 2017



Edward Hance Shortliffe

b. 1947

2. Why Bayesian netwoks

What do graphical probabilistic models offer when we already have powerful neural predictors?

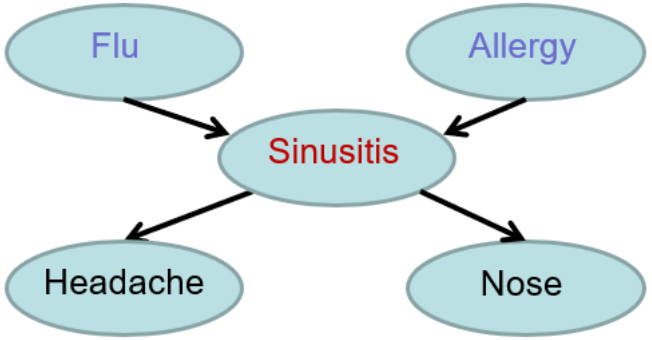
- Represent uncertain knowledge with explicit dependencies
- Provide structured assumptions and conditional queries
- Update beliefs when new evidence arrives
- Provide probabilistic reasoning components for hybrid AI systems
- Are readable models. Important for clinical decision support, reliability, risk. The EU AI Act asks high-risk systems for transparency and human oversight; a BN is auditable by design

Three tasks: representation, inference, learning

- **Representation.** How do we express/represent a probability distribution that models some real-world phenomenon?
  - Construct a DAG and local conditional probability distributions
- **Inference.** Given a probabilistic model, how do we obtain answers to relevant questions about the world?
  - **Marginal inference** – query the JPD; *compute P(Query | Evidence)*
  - **Maximum a posteriori (MAP) inference** – asks for the most likely assignment of variables; $argmax_Y P(Y|x_1,..,x_n)$
- **Learning.** Fitting a model to a dataset
  - By looking at the data, how can we infer the parameters or the structure of a BN?

Learning and inference are inherently linked since inference is a key subroutine that is used in BN learning algorithms

Example



F – Flu
A – Allergy
S – Sinusitis
H – Headache
N – Nasal symptoms

Conditional Probability Table (CPT)

P(F=1)=0.4

P(A=1)=0.3

P(S=1|F,A):
F=0,A=0 = 0.05
F=0,A=1 = 0.10
F=1,A=0 = 0.60
F=1,A=1 = 0.80

P(H=1|S=1)=0.9
P(H=1|S=0)=0.2

P(N=1|S=1)=0.85
P(N=1|S=0)=0.25

- **Representation**: Simplifying assumptions: F ⊥ A and H ⊥ N | S
  Both assumptions are modeling choices that may be false in reality
- **Inference**: P(Sinusitis =1 | Flu=1, Allergy=0)
- **Learning**: Learn CPTs from data based on a dataset of patients

Refresh your knowledge

Product rule

$$P(A|B) = P(A \wedge B) / P(B), \quad P(B)>0$$

Chain rule (General form of Product rule)

$$P(A_1 \wedge \ldots A_n) = P(A_1) * P(A_2|A_1) * \ldots * P(A_n|A_1 \wedge \ldots \wedge A_{n-1})$$

Conditioning Rule (Rule of total probability)

$$P(A) = P(B_1) * P(A|B_1) + P(B_2) * P(A|B_2) +. . .+ P(B_n) * P(A|B_n)$$

Conditional independence of 2 events

- An event **A** is independent of an event **B** if

$$P(A|B) = P(A) \quad \text{or} \quad P(A \wedge B) = P(A) * P(B)$$

Notation $P \models (A \perp B)$

Conditional independence of 2 events given one additional event

- An event **X** is independent of **Y** given **Z** if

$$P(X|Y \wedge Z) = P(X|Z) \quad \text{or} \quad P(X \wedge Y|Z) = P(X|Z) * P(Y|Z)$$

Notation $P \models (X \perp Y \mid Z)$

Refresh your knowledge

- **Marginalization** $P(X) = \sum_{y} P(X, y)$
- **Bayes's theorem** $P(A|B) = \frac{P(B|A) * P(A)}{P(B)}$
- **Conditionalized version of Bayes's theorem** $P(A|B, C) = \frac{P(B|A, C) * P(A|C)}{P(B|C)}$
- **Bayes theorem with several hypotheses** $P(h_i|e) = \frac{P(e|h_i) * P(h_i)}{\Sigma_{j=1}^{k} P(e|h_j) * P(h_j)} \quad i = 1, k$
- **Bayes theorem – general form** $P(h_i|e_1, e_2, \ldots, e_n) = \frac{P(e_1, e_2, \ldots, e_n|h_i) \cdot P(h_i)}{\sum_{j=1}^{k} P(e_1, e_2, \ldots, e_n|h_j) \cdot P(h_j)}, \quad i = 1, k$

3. Bayesian inference - Marginal

Marginal inference (JPD queries)

- How to use a JPD (Joint Probability Distribution) over multiple random variables to answer queries of interest

Consists of 2 parts

- The ***evidence*** – a subset **E** of random variables and an instantiation **e** of these variables
- The **query variables** – a subset **X** of random variables
- The task is to compute **P**(**X**|**E**=e)

General inference based on JPD

Query X

X – query variable

E – evidence variables

e – values observed for evidence variables E

Y – variables that were not observed

$\mathbf{P}(X \mid e) = \alpha * \mathbf{P}(X, e) = \alpha * \Sigma_Y \mathbf{P}(X, e, Y)$

Example: Marginal

Three binary variables **F = Flu, S = Sinusitis, H = Headache**

What is the value of **P(S = 1 | H = 1)**? —
Probability of sinusitis given headache

JPD

F	S	H	P(F, S, H)
0	0	0	0.30
0	0	1	0.10
0	1	0	0.05
0	**1**	**1**	**0.15**
1	0	0	0.10
1	0	1	0.05
1	1	0	0.05
1	**1**	**1**	**0.20**
Total probability = 1.00			

$$\mathbf{P(X \mid e) = \alpha \Sigma_Y P(X, e, Y)}$$

X = S , e = (H = 1) , Y = F

$$P(S=1 \mid H=1) = \alpha \Sigma_{F=0 \text{ to } 1} P(S=1,H=1,F)$$

Sum out Flu (F)

0.15 + 0.20 = 0.35

Normalize using P(H=1)

0.10 + 0.15 + 0.05 + 0.20 = 0.50

P(S=1 | H=1) = 0.35 / 0.50 = 0.70

Bayesian inference – MAP

Maximum a posteriori query (MAP inference)

Find the value of Y that maximizes the **posterior probability Y** given the evidence **$X = x_1,..,x_n$**

$$MAP(\mathbf{Y}|\mathbf{X}) = \text{argmax}_{\mathbf{Y}} P(\mathbf{Y}|\mathbf{X}) = \text{argmax}_{\mathbf{Y}} P(\mathbf{Y}|x_1,..,x_n)$$

MAP finds the **most probable value** of the unknown variable that best explains the evidence

Can be expressed as

$$\mathbf{MAP(Y=y|x_1,..,x_n)} = \text{argmax}_{\mathbf{Y}} P(Y|x_1,..,x_n) = \text{arg max}_{y} [P(x_1,..,x_n|y) * P(y) / P(x_1,..x_n)]$$

$$= \text{arg max}_{y} [\alpha * P(x_1,..,x_n|y) * P(y)] = \mathbf{argmax}_{\mathbf{y}} \mathbf{[P(y) * P(x_1,\ldots,x_n|y]}$$

For the previous example, given that a patient has both Flue and Headache, is it more probable that the patient has Sinusitis or does not have Sinusitis?

$$MAP(\text{Sinusitis} \mid \text{Flue, Headache}) = \text{arg max}_S P(S=s \mid F = 1, H = 1)$$

Naïve Bayes assumption

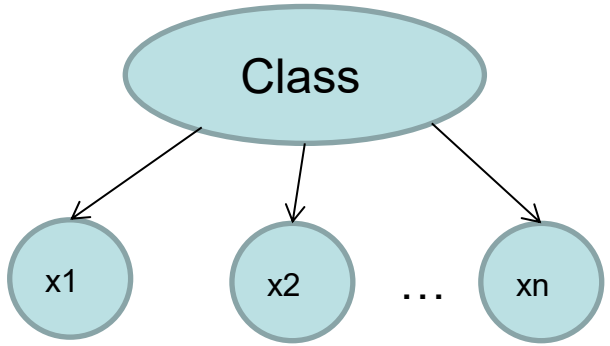
If $X_1..X_n$ are conditionally independent given Y then $P(X_1,..X_n|Y) = \Pi_i P(X_i|Y)$

$MAP(Y=y|x_1,..,x_n) = \arg\max_y [P(y) * \Pi_i P(x_i|y)]$

Can be also used to perform classification

Feature vector $x_1,..,x_n$ and a set of classes C_k → **Naïve Bayes classification**

$C_MAP = \text{argmax}_k \mathbf{P}(C_k) * \Pi_i \mathbf{P}(x_i|C_k)$



provided $x_1...x_n$ are conditionally independent given C_k

Marginal vs. MAP queries

$P(Y|X_1=x_1, .. X_n=x_n)$

- The output of a JPD query is a **specific probability value or** a **probability distribution** showing how likely different values of Y are, given the evidence X
- This query answers questions like: "*Given this evidence, what is the probability that Y takes each possible value*?"

$MAP(Y|X) = \arg\max_Y P(Y|X_1=x_1, \ldots X_n=x_n)$

- The output of a MAP query is **an assignment of values to a variable** or **a set of variables $Y_1, .. Y_m$** that maximizes the posterior probability of Y based on evidence variables X
- This query answers questions like: "*What is most probable value that best explains the evidence X*"

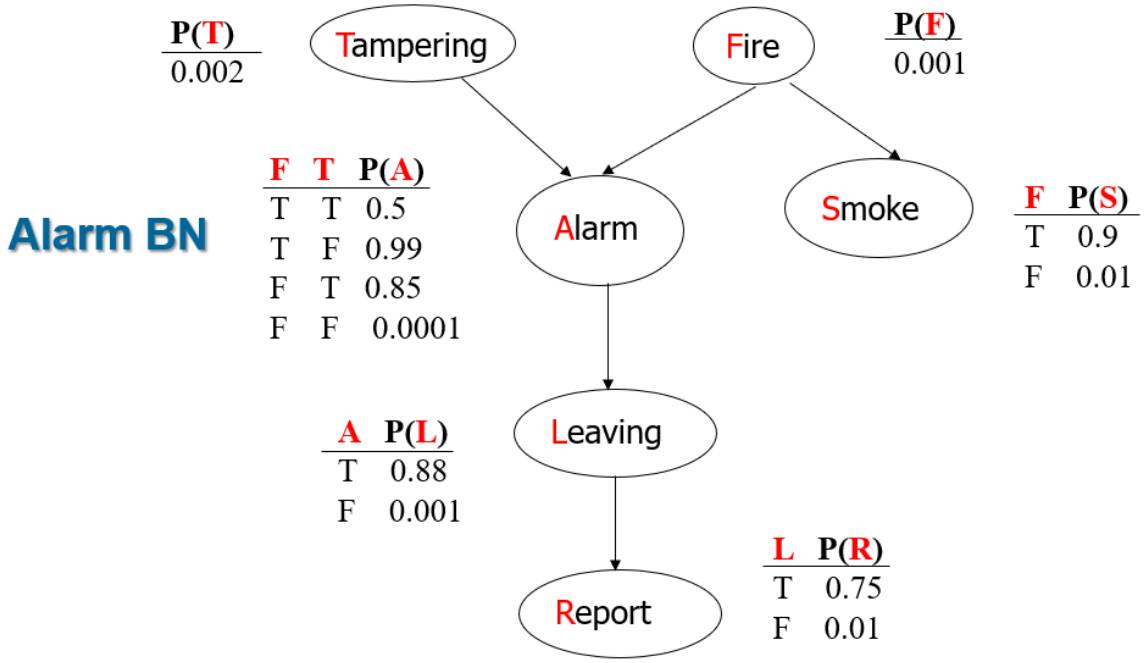
4. Bayesian networks

- **Bayesian Network idea:** independence properties in a JPD can be used to represent high-dimensional distributions more compactly
- A BN (Belief Network or Bayesian Network) is a graphical representation of conditional independence
- It is represented a **Directed Acyclic Graph** (DAG)
- The nodes represent random variables
- The edges represent direct dependence among the variables
- **X→Y**: X has a direct influence on Y (represents a statistical dependence)
- **X = Parent(Y)** if X→Y
- Each node has a probability distribution table indicating how the probability of this variable's values depends on all possible combinations of parent values

$$\mathbf{P(X=a|Y_1=v_1 \wedge \ldots \wedge Y_n=v_n)}$$

A Bayesian network

- A BN defines the **JPD** over the variables in the network



- A BN is a correct representation of the domain, provided that each node is conditionally independent of its non descendants, given its parents

that is

$\mathbf{P \models (X_i \perp NonDesc_{Xi} \mid Parents_{Xi})}$, ∀i

- The graph structure **G** of a BN encodes a certain set of conditional independence assumptions
- Some can be seen directly, some others can be inferred – **D-separation**

Instead of computing JPD of all the variables by the chain rule
P(T,F,A,S,L,R) =
*P(T)*P(F|T)*P(S|F,T)*P(A|S,F,T)*P(L|A,S,F,T)*P(R|L,A,S,F,T)*

We use JPD in the Bayesian network

P(T,F,A,S,L,R) = P(T) * P(F) * P(S|F) * P(A|T,F) * P(L|A) * P(R|L)

D-separation

- When the property **(X ⊥ Y | Z)** does not hold?"

Direct connection of 2 nodes X→Y

- They can influence each other regardless of Z → **(X ⊥̸ Y | Z)**

Indirect connection: X and Y are not directly connected but there is a trail (a path not considering arrows direction) between them in graph

When influence can flow from X to Y via Z we say the trail X ↔ Z ↔ Y is active

Causal trails X → Z → Y – active iff Z is not observed

Evidential trails Y → Z → X – active iff Z is not observed

Common cause X ← Z → Y – active iff Z is not observed

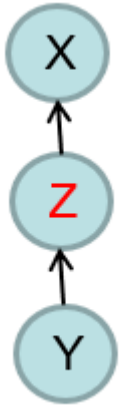
Common effect X → Z ← Y - active iff either Z or one of its descendants are observed

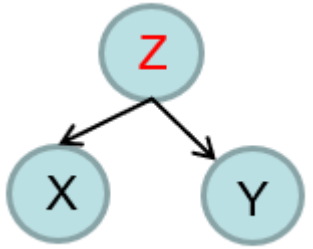
The structure **X → Z ← Y** is called a collider

(X ⊥ Y | Z) if Z observed

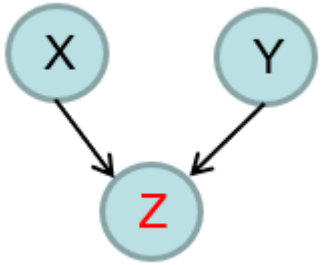


(Y ⊥ X | Z) if Z observed





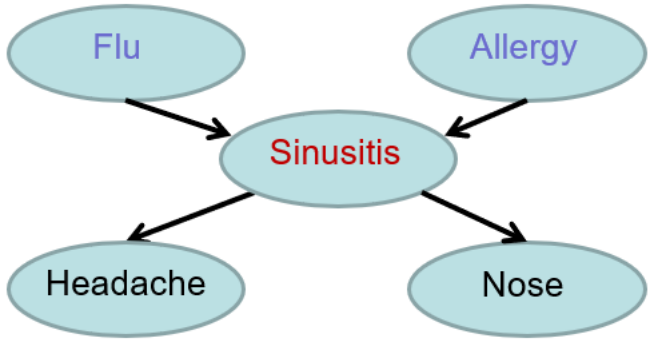
(X ⊥ Y | Z) if Z observed



(X ⊥ Y | Z) if Z not observed

Example

- $F \rightarrow S \leftarrow A$ is a collider
- Before observing S, $F \perp A \mid S$
- After observing S=1, evidence for F can reduce the posterior probability of A, therefore $F \not\perp A \mid S$
- This is explaining away
- Before observing S, $H \not\perp N \mid S$
- After observing S=1, $H \perp N \mid S$
- H and N are related only because they share a cause. Once that cause is known, nothing else connects them, so observing S blocks the path between them

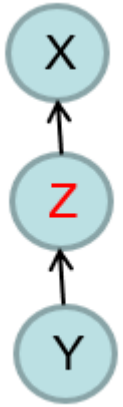


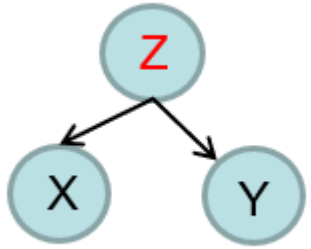
F – Flu
A – Allergy
S – Sinusitis
H – Headache
N – Nasal symptoms

$(X \perp Y \mid Z)$ if Z observed

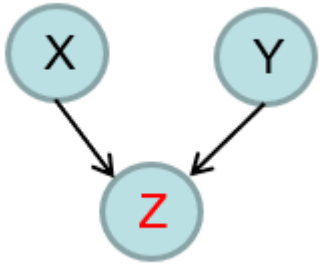


$(Y \perp X \mid Z)$ if Z observed





$(X \perp Y \mid Z)$ if Z observed



$(X \perp Y \mid Z)$ if Z not observed

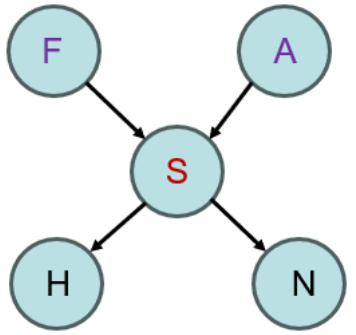
Inference queries in BN

- **Marginal**

$$P(S=1) = \sum_{F=0}^{1} \sum_{A=0}^{1} P(F)\, P(A)\, P(S=1 \mid F, A)$$

- **Conditional**

$$P(S=1 \mid H=1, N=1)$$



- **MAP** - *Maximum a posteriori* : Finds the most probable assignment of selected query variables. Other unobserved variables are marginalized out

$$MAP = \arg\max_s P(S=s \mid H=1, N=1)$$

- **MPE** - *Most Probable Explanation*: most probable joint assignment of all unobserved variables, given the observed evidence. All unobserved variables are assigned values

Example: Suppose we observe that a patient has sinusitis (S=1), but we do not know whether the patient has flu or allergy. MPE determines the most probable combination of values for Flu and Allergy, given the observed sinusitis

$$MPE = \arg\max_{F, A} P(F, A \mid S=1)$$

MPE is a special case of MAP inference in which the query includes all unobserved variable

5. Inferences in Bayesian Networks

- Aim: compute ***probabilities queries (marginal inference)*** but then can be used for other types of queries

Exact inference algorithms

- **Variable elimination algorithm**
- **Junction tree algorithm (or clique trees)**

Approximation inference algorithms

- Sampling - stochastic approximation
- Variational inference

5.1 Variable elimination algorithm

- Variable elimination algorithm efficiently computes the marginal **P(X | e)**
- A smart way to ***multiply and sum probabilities*** without enumerating every possible variable configuration

The VE algorithm repeatedly performs two operations:

- **factor product**
- **marginalization (marginalization factor)**

- The **factor product** operation defines the product $\phi_3 = \phi_1 \times \phi_2$ of two factors ϕ_1 and ϕ_2

$$\phi_3(a, b, c) := \phi_1(a, b) \times \phi_2(b, c)$$

- The scope of ϕ_3 is defined as the union of the variables in the scopes of ϕ_1,ϕ_2

- The **marginalization operation** "locally" eliminates a set of variable from a factor
- If we have a factor $\phi(X,Y)$ over two sets of variables **X**,**Y**, marginalizing **Y** produces a new factor $\tau(X)$, called **marginalized factor**

$$\tau(x) = \sum_y \phi(x, y)$$

where the sum is over all joint assignments to the set of variables **Y**

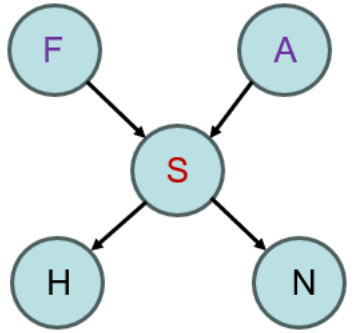
- This factor does not necessarily correspond to a probability distribution, even if ϕ was a CPD

Example: Marginal inference using VE

$$P(F, A, S, H, N) = P(F)P(A)P(S \mid F, A)P(H \mid S)P(N \mid S)$$

Factors

$$\phi_1(F) = P(F)$$
$$\phi_2(A) = P(A)$$
$$\phi_3(S, F, A) = P(S \mid F, A)$$
$$\phi_4(H, S) = P(H \mid S)$$
$$\phi_5(N, S) = P(N \mid S)$$



P(F=1)=0.4

P(A=1)=0.3

P(S=1|F,A):
F=0,A=0 = 0.05
F=0,A=1 = 0.10
F=1,A=0 = 0.60
F=1,A=1 = 0.80

P(H=1|S=1)=0.9
P(H=1|S=0)=0.2

P(N=1|S=1)=0.85
P(N=1|S=0)=0.25

- **Marginal inference using VE**: What is the probability of sinusitis before observing any symptoms? P(S = 1) = ?

$$P(S) = \sum_{F=0}^{1} \sum_{A=0}^{1} P(F)\, P(A)\, P(S \mid F, A)$$

Eliminate F $\quad \tau_1(A, S) = \sum_{F=0}^{1} P(F)P(S \mid F, A)$

$\tau_1(\mathrm{A}{=}0, \mathrm{S}{=}1) = \mathrm{P}(\mathrm{F}{=}0)\cdot\mathrm{P}(\mathrm{S}{=}1 \mid \mathrm{F}{=}0, \mathrm{A}{=}0) + \mathrm{P}(\mathrm{F}{=}1)\cdot\mathrm{P}(\mathrm{S}{=}1 \mid \mathrm{F}{=}1, \mathrm{A}{=}0) = 0.6{\cdot}0.05 + 0.4{\cdot}0.60 = 0.03 + 0.24 = \mathbf{0.27}$

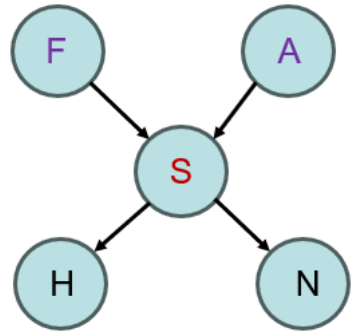
$\tau_1(\mathrm{A}{=}1, \mathrm{S}{=}1) = \mathbf{0.38}$

Eliminate A $\quad \tau_2(S) = \sum_{A=0}^{1} P(A)\tau_1(A, S)$

$\mathbf{P(S = 1) = 0.7 \cdot 0.27 + 0.3 \cdot 0.38 = 0.303}$

Example: Conditional inference using VE

- **Conditional inference using VE**: What is the probability of sinusitis if the patient has headache and nasal symptoms? $P(S = 1 \mid H = 1, N = 1) = ?$



P(F=1)=0.4

P(A=1)=0.3

P(S=1|F,A):
F=0,A=0 = 0.05
F=0,A=1 = 0.10
F=1,A=0 = 0.60
F=1,A=1 = 0.80

P(H=1|S=1)=0.9
P(H=1|S=0)=0.2

P(N=1|S=1)=0.85
P(N=1|S=0)=0.25

Incorporate the evidence
Set H=1 and N=1 in their factors

$$\phi_4(S) = P(H = 1 \mid S) \qquad \phi_5(S) = P(N = 1 \mid S)$$

Eliminate F and A by summation $\tau(S) = \sum_F \sum_A P(F)P(A)P(S \mid F, A)$

We already know $\tau(S = 1) = 0.303, \quad \tau(S = 0) = 0.697$

Multiply by the evidence factors $g(S) = \tau(S)P(H = 1 \mid S)P(N = 1 \mid S)$ pointwise product for each value of S

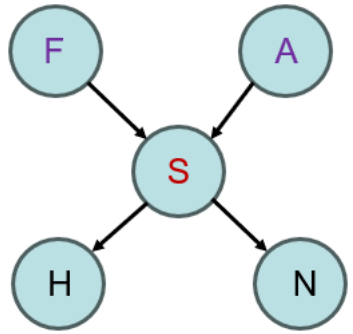
$$g(S = 1) = 0.303(0.9)(0.85) = 0.231795 \qquad g(S = 0) = 0.697(0.2)(0.25) = 0.03485$$

Normalize $P(S = 1 \mid H = 1, N = 1) = \frac{g(1)}{g(1) + g(0)}$ $\mathbf{P(S = 1 \mid H = 1, N = 1) \approx 0.8693}$

The probability of sinusitis increases from 30.3% to approximately 86.9% after observing the two symptoms

Example: MAP inference using VE

- **MAP inference using VE**: Given headache and nasal symptoms, what is the most probable value of Sinusitis?



$$S_{\text{MAP}} = \arg\max_S P(S \mid H = 1, N = 1)$$

Incorporate evidence: Set H=1,N=1

Sum out F and A

$$g(S) = \sum_F \sum_A P(F)P(A)P(S \mid F, A)P(H = 1 \mid S)P(N = 1 \mid S)$$

Maximize over S $\quad S_{\text{MAP}} = \arg\max_S g(S) \quad S_{\text{MAP}} = 1$

The most probable diagnosis is that the patient has sinusitis

5.2 Junction Trees algorithm

- VE algorithm can answer queries of the form $\mathbf{P(Y_1|E=e_1)}$
- If we want to ask the model for another query, e.g. $\mathbf{P(Y_2|E_2=e_2)}$, we need to restart the algorithm from scratch
- When computing marginals, VE produces many intermediate factors τ as a side-product of the main computation
- These factors are the same as the ones that we need to answer other marginal queries
- By **caching them** after a first run of VE, we can easily answer new marginal queries at almost no additional cost

Main idea

Decompose global calculations of joint probability into a **linked set of local computations**

Build a data structure that can answer a large class of queries without additional computation (almost)

Steps

1. **Moralize the graph**
2. **Triangulate** the graph and cluster variables into **cliques (junctions)**
3. Transform these cliques into a **tree of cliques** (junctions) – **Junction tree**
4. Perform **belief propagation** (**message passing**) on this tree

Junction Trees

- Each **node** in a cliques graph / clique tree represents a **group of variables** (a subset $\mathbf{x_c}$ of G nodes) from the original BN graph
- These groups are called ***cliques*** *or* ***junctions*** and are connected in a special way
- In a clique graph, we label an edge with the variables from the intersection of the two cliques it connects – **separation sets** or "**sepsets**"

A **junction tree / clique tree T** of the clique graph:

- Is a tree (singly connected)
- Contains all the cliques (spanning tree)
- It satisfies the following properties:
  - **Family preservation** – for each factor $\phi(\mathrm{x_c})$ of the Bayesian network there is at least one clique (cluster) which contains all the variables of the factor
  - **Running intersection** – for each pair of clusters U, V, all nodes on the path between U and V contain U ∩ V

Each clique is assigned a function $\phi(\mathrm{C_i})$, the **clique potential** = the product of its assigned factors

- *Family Preservation ensures that every original factor can be assigned to a clique, enabling the construction of clique potentials*
- *Family Preservation tells us where factors can be placed; Clique Potentials tell us what numerical information each clique contains*

An example of building a junction tree

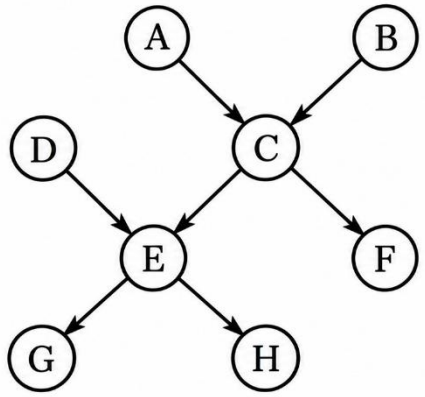
a) Moralize the graph
b) Triangulate the graph
c) Create the junction graph
d) Create the junction tree

The computational cost of inference depends primarily on the size of the largest clique
Optimal trees - make the clusters as small as possible; it is NP-hard to find the optimal tree.

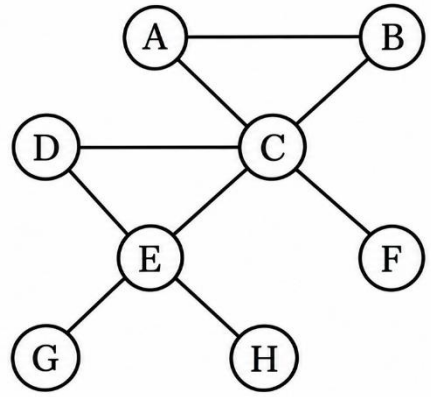
If the graph itself is a tree - define a cluster for each edge in the tree

$$A - B - C - D$$

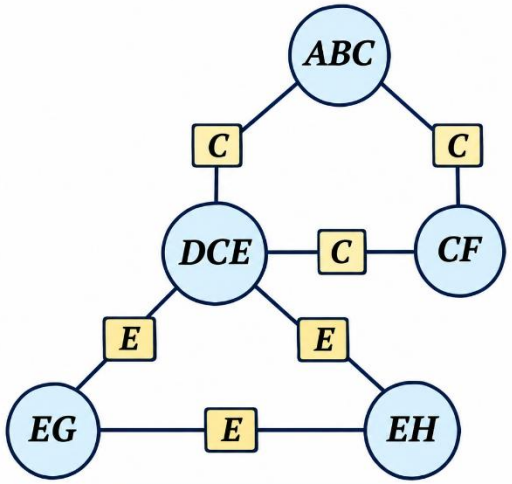
$$C_1 = \{A, B\} \qquad S_{12} = \{B\}$$
$$C_2 = \{B, C\} \qquad S_{23} = \{C\}$$
$$C_3 = \{C, D\}$$



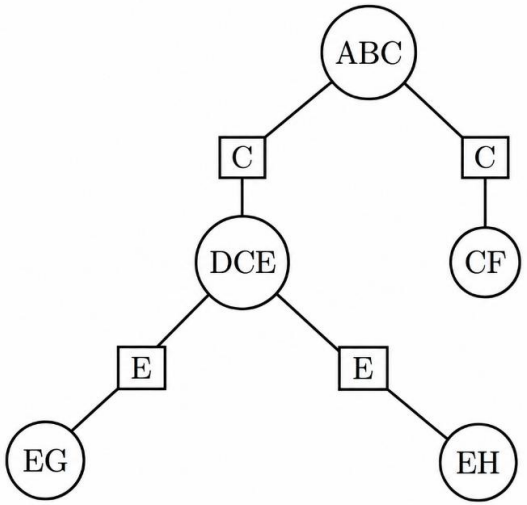
(a) DAG



(b) Moral Graph



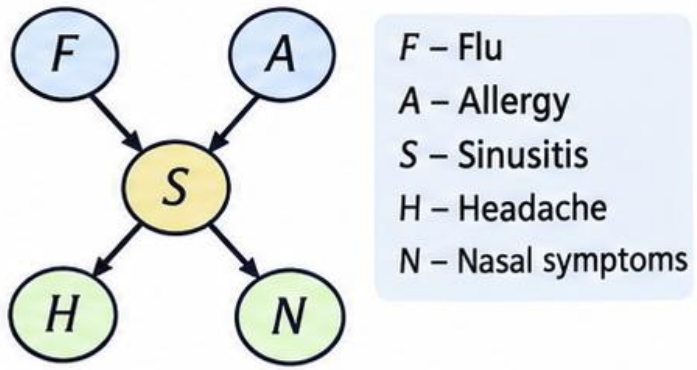
(c) Junction Graph



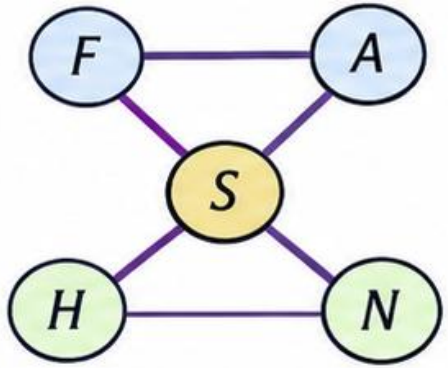
(d) Junction tree

The Junction tree for our example

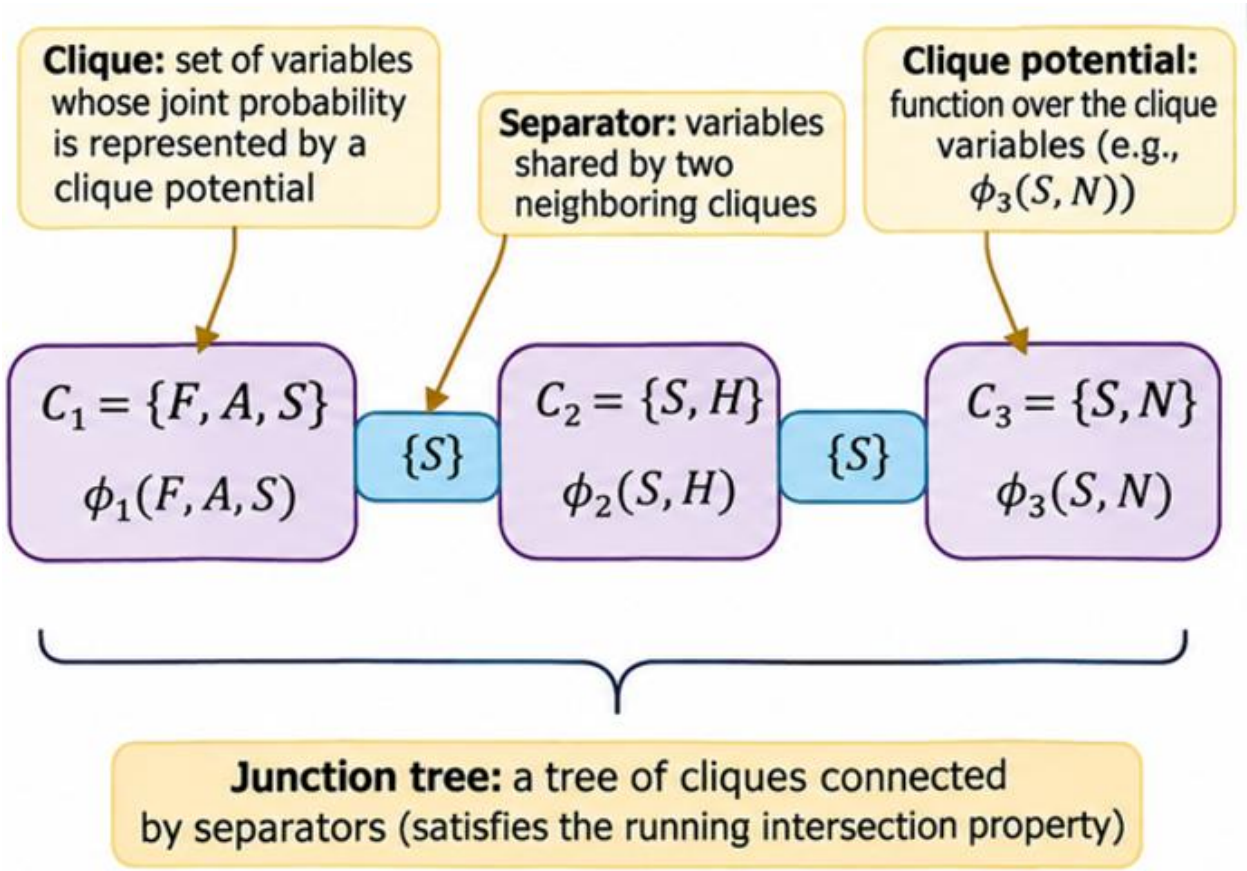
Bayesian network



Triangulated graph



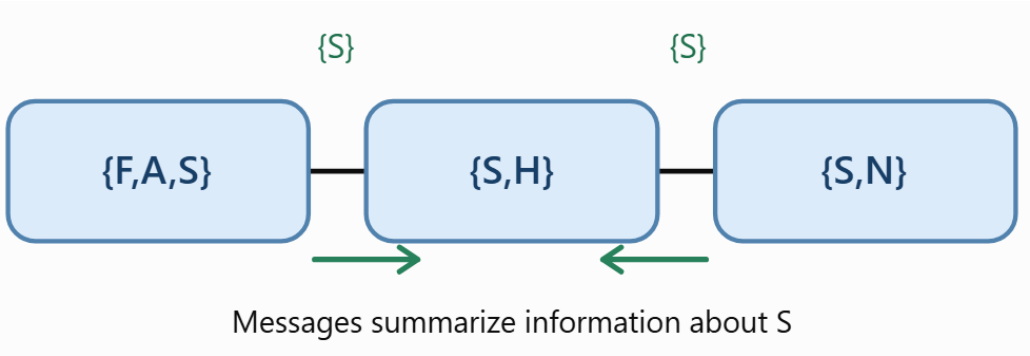
Junction tree



Belief propagation

Idea of belief propagation: Variable elimination as message passing

- Performing probabilistic inference by exchanging messages between the cliques of a Junction Tree
- Instead of computing probabilities using the entire Bayesian Network at once, **each clique performs local calculations and communicates the relevant results to neighboring cliques**



- Each clique has its potential
- A clique sends a message to a neighboring clique by combining its potential with incoming messages and summing out variables not shared with that neighbor
- Each message is a function of the separator variables
- After message passing, each clique combines its potential with incoming messages to obtain its belief (marginal distribution after normalization)

Compute the probability of Sinusitis, **P(S)**

- Clique {F,A,S} summarizes the information about Flu and Allergy and sends a message about S
- Clique {S,N} summarizes the information about Nasal Symptoms and sends a message about S
- Clique {S,H} combines these messages with its own information about Headache to compute the probability of S

Belief propagation: General case

General case: the complete belief propagation process

- Message passing has **two passes**
  - Collect Evidence phase (upward pass): Gather information from the entire network at a selected root clique
    Each node gathers evidence from its neighbors and sends a message to its parent
  - Distribute Evidence phase (downward pass): Propagate this information back to all other cliques.
    Once the root has collected all messages, it sends back messages **downward** so all cliques become consistent
- Result: Every clique has a consistent marginal distribution that incorporates all observed evidence

Belief propagation - comments

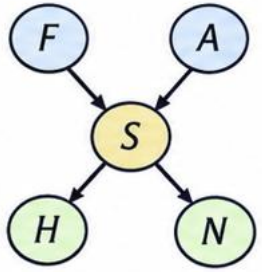
- To answer one query, collecting messages toward an appropriate clique may be sufficient

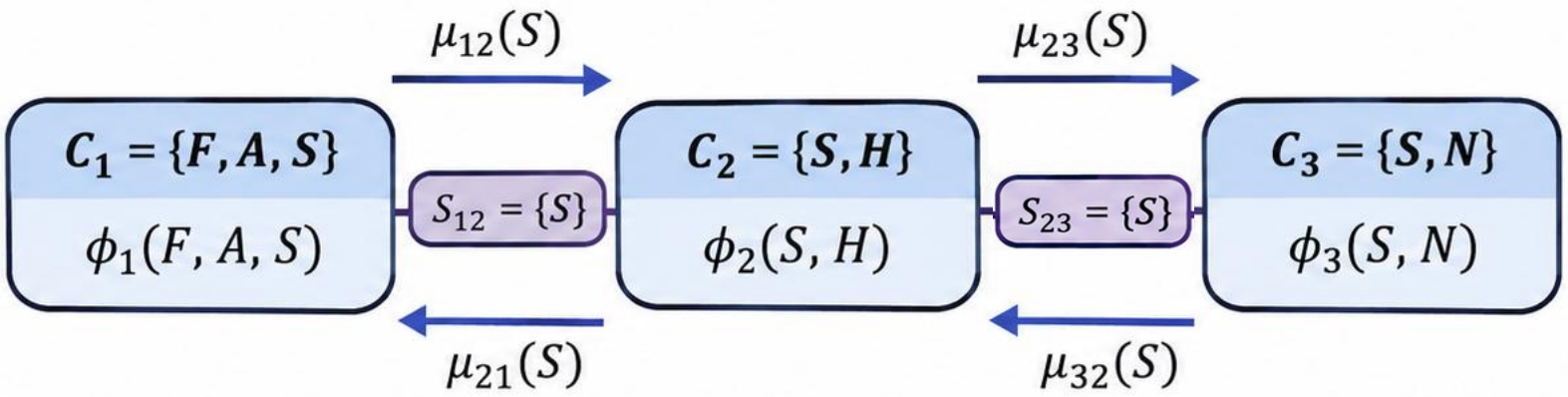
- **Complete belief propagation**: Multiple probabilistic queries can be answered efficiently without repeating the complete inference process

- Bidirectional message passing makes probabilistic information available throughout the Junction Tree, not only at the selected root clique

Note: if new evidence is introduced, the junction tree must be updated; the previously computed marginals are not automatically valid for the new evidence

Example of Belief propagation





- **Clique potential** $\phi_i(C_i)$: joint probability of the variables in clique C_i
- **Message** $\mu_{ij}(S)$: a function over the separator S (shared variable S)
- When a clique sends a message, it summarizes (marginalizes) its information about the shared variable S.

Compute P(S)

Initialize clique potentials: $\phi_1(F, A, S)$, $\phi_2(S, H)$, $\phi_3(S, N)$

Pass messages: $\mu_{12}(S)$ from $C_1 \rightarrow C_2$ and $\mu_{32}(S)$ from $C_3 \rightarrow C_2$

Compute belief (marginal) at C_2: $b_2(S) = \phi_2(S) \cdot \mu_{12}(S) \cdot \mu_{32}(S)$

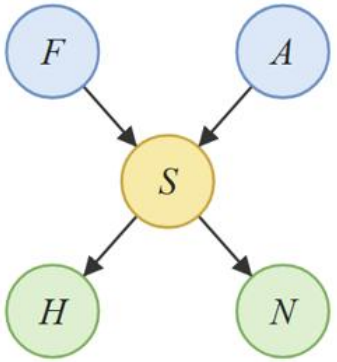
Normalize $b_2(S)$ to obtain $P(S)$.

6 Complexity of inferences in BN

Exact Inference in Bayesian Networks

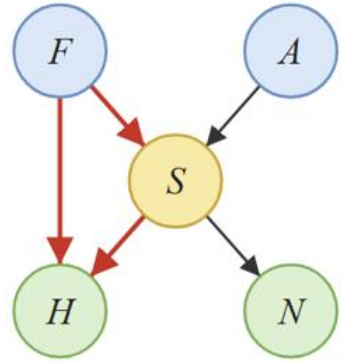
- **Inference by enumeration**
  - $\mathrm{P}(\mathrm{X} \mid \mathrm{e}) = \alpha \, \Sigma_{\mathrm{y}} \, \mathrm{P}(\mathrm{X}, \mathrm{e}, \mathrm{y})$
  - Space: O(n). Time: $O(2^n)$ for n Boolean variables
- **Variable elimination**
  - Avoids repeated computations by storing intermediate factors
  - **Polytrees** (at most one undirected path between any two nodes): linear time and space
  - **Multiply connected networks**: exponential in the worst case
- **Junction (clique) tree algorithm**
  - Clusters nodes into cliques so that the network becomes a tree
  - Message passing is linear in the size of the tree and gives all posteriors in one run
  - Clique tables grow exponentially with the **treewidth**
- **In general:** exact inference in BNs is NP-hard, so for large, densely connected networks we turn to **approximate inference**

Polytree



at most one path between any two nodes

Not a polytree



two paths between F and H
F→S→H and F→H (undirected loop)

6 Approximate Inference in BN

- **Sampling-based inference**
  - Random sampling algorithms, also known as Monte Carlo algorithms
  - They provide approximate answers whose accuracy depends on the number of samples generated
  - The idea is to sample each variable in turn, in topological order
  - Samples are generated from a known probability distribution

- **Optimization-based inference**
  - Optimization-based (variational) inference algorithms, which turn inference into an optimization problem
  - They approximate the true posterior with a simpler, tractable distribution

Sampling: Rejection sampling

Rejection Sampling

- Estimate a conditional probability P(X | e)
- Samples are generated from the probability distribution specified by the network
- Then the samples that do not match the evidence are rejected
- Finally, the estimate of P(X = x | e) is obtained by counting how often X = x occurs among the remaining samples

Limitation: wastes many samples when evidence is rare



P(S=1) = 0.30

P(H=1 | S=1) = 0.90

P(H=1 | S=0) = 0.20

P(S=1 | H=1)

1. Generate complete random cases (S, H) from the BN
2. Reject cases with H=0
3. Among the remaining cases, count those with S=1

Illustrative simulation

405 kept, 270 also have S=1 → 270 / 405 ≈ 0.67

Sampling: Likelihood Weighting

Likelihood Weighting

- It avoids the potential inefficiency of rejecting samples by generating only those samples that are consistent with the evidence
- The values of the evidence variables E are fixed, and only the remaining variables are sampled
- This guarantees that every generated sample is consistent with the evidence, but not all events are equally important
- Before being counted, each event is weighted by the likelihood that the sample assigns to the evidence

Advantage: no samples are rejected. More samples improve accuracy



P(S=1) = 0.30

P(H=1 | S=1) = 0.90

P(H=1 | S=0) = 0.20

P(S = 1 | H = 1)

1. Fix the observed variable H=1
2. Sample only S from its prior distribution P(S), with H=1 fixed
3. Assign weight P(H=1 | S)

Sampled S	Weight	Contribution to S=1
1	0.90	0.90
0	0.20	0
1	0.90	0.90
0	0.20	0

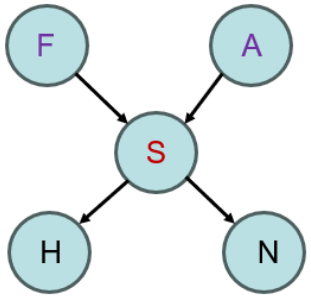
Weights of samples S = 1 is 0.90 + 0.90 = 1.80
Weights of all sampes is 0.90 + 0.20 + 0.90 + 0.20 = 2.20

Estimate = 1.80 / 2.20 ≈ 0.82 (only 4 samples)

Variational Inference (VI)

- Optimization-based inference
- Approximate a difficult posterior with a simpler distribution

Return to the full Sinusitis BN



Evidence: H=1, N=1

Posterior to approximate:

P(F, A, S | H=1, N=1)

MAIN IDEA

1. Choose an easier distribution, e.g.

q(F, A, S) = q(F) q(A) q(S)

2. Adjust q to approximate the true posterior

(minimize KL(q || posterior), or maximize ELBO)

Example output: q(S=1) ≈ 0.86

Note: A Variational Autoencoder (VAE) is a neural latent-variable model trained using the principles of Variational Inference.

Sampling methods vs. Variational Inference

Aspect	Sampling methods	Variational Inference
Main idea	Generate random samples	Optimize a simpler distribution
Result	Estimates from samples	An approximate probability distribution
Computation	Repeated sampling	Iterative optimization
Main limitation	Sampling may be slow or inefficient	Approximation may introduce systematic bias
Typical methods	Gibbs sampling, importance sampling	Mean-field VI, structured VI

Annexes

Annex 1: Examples for probability rules

Product rule

- Toss two fair coins, blindfolded. Somebody tells you that you tossed at least one Heads. What is the probability that both tosses are Heads?

A = {both H}, B = {at least one H}

$P(A|B) = P(A \wedge B) / P(B) =$ P(both H) / P(at least one H) = (1/4) / (3/4) = 1/3

Conditioning rule

- Flip a fair coin. If you toss Heads, roll 1 die. If you toss Tails, roll 2 dice. Compute the probability that you roll exactly one 6.
- Here you condition on the outcome of the coin toss, which could be Heads (event F) or Tails (event F^c)

If A = {exactly one 6}, then $P(A|F) = 1 / 6$, $P(A|F^c) = 5 / 36 + 5 / 36$

$P(F) = P(F^c) = ½$

$P(A) = P(F)*P(A|F) + P(F^c)*P(A|F^c) = 2 / 9$

Annex 2: Example of using Bayes Theorem

A factory has three machines, M1, M2 and M3, that produce items. It is impossible to tell which machine produced a particular item, but some are defective.

machine	proportion of items made	prob. any made item is defective
M1	0.2	0.001
M2	0.3	0.002
M3	0.5	0.003

Pick an item, test it, and find it is defective. What is the probability that it was made by machine M2?

First choose a machine with the probabilities given by the proportion.

Then, that machine produces an item, which you then proceed to test.

D - event that an item is defective

M_i - event that the item was made by machine **i**.

$$P(M_2|D) = \frac{P(D|M_2) * P(M_2)}{\sum_i (P(D|M_i) * P(M_i))}$$

P(D|M1) = 0.001, P(D|M2) = 0.002, P(D|M3) = 0.003

P(M1) = 0.2, P(M2) = 0.3, P(M3) = 0.5

P(M2|D) = (0.002 * 0.3) / (0.001 * 0.2 + 0.002 * 0.3 + 0.003 * 0.5) ≈ 0.26

Annex 3: D-separation

When influence can flow from X to Y via Z we say the trail $X \leftrightarrow Z \leftrightarrow Y$ is <u>active</u>

- Causal trails $X \rightarrow Z \rightarrow Y$ – active iff Z is not observed
- Evidential trails $Y \rightarrow Z \rightarrow X$ – active iff Z is not observed
- Common cause $X \leftarrow Z \rightarrow Y$ – active iff Z is not observed
- Common effect $X \rightarrow Z \leftarrow Y$ - active iff either Z or one of its descendants are observed
  - The structure $X \rightarrow Z \leftarrow Y$ is called a v-structure
- **Probabilistic influence** = **flow in graph** - when influence from X can "***flow***" through Z to affect our belief about Y

D-separation – General case

Longer trail $\mathbf{X_1 \leftrightarrow \ldots \leftrightarrow X_n}$

- To flow from X_1 to X_n it needs to flow through every single node on the trail
- X_1 can influence X_n if every two-edge trail $\mathbf{X_{i-1} \leftrightarrow X_i \leftrightarrow X_{i+1}}$ along the trail allows influence to flow

Let G be a belief network structure and $X_1 \leftrightarrow \ldots \leftrightarrow X_n$ a trail in G. Let Z be a subset of observed variables. The **trail $\mathbf{X_1 \leftrightarrow \ldots \leftrightarrow X_n}$ is active given Z** if:

- Whenever there is a v-structure $\mathbf{X_{i-1} \rightarrow X_i \leftarrow X_{i+1}}$ then X_i or one of its descendents are in Z
- No other node along the trail is in Z

- Note: If X_1 or X_n are in Z the trail is not active

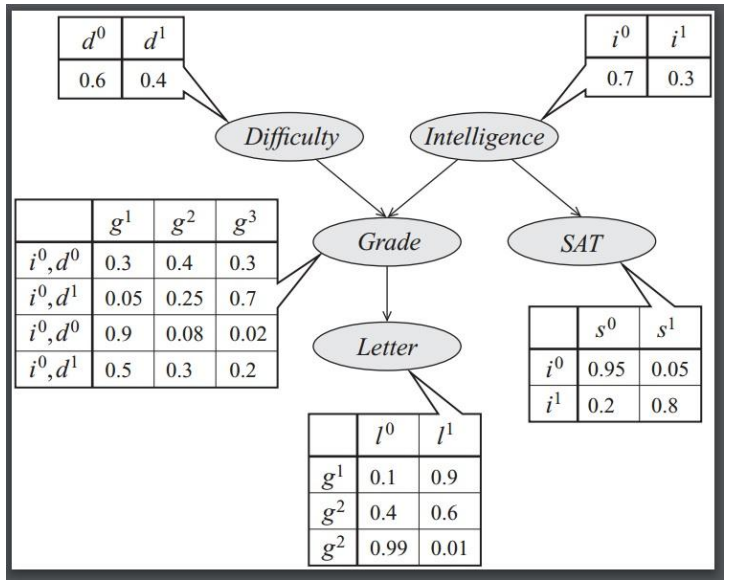
D-separation – Example

- **D → G ← I** with G observed

The probe that the student has high intelligence goes down when we observe that his grade is C but then goes up when we observe that the class is difficult

- **D → G ← I** is not active for G not observed because the v-structure **D → G ← I** is not activated
- The same trail is active when **L** is observed because observing the descendant of **G** activates the v-structure
- When **L and I** are observed the trail is not active because observing **I** blocks the trail **G ← I → S**

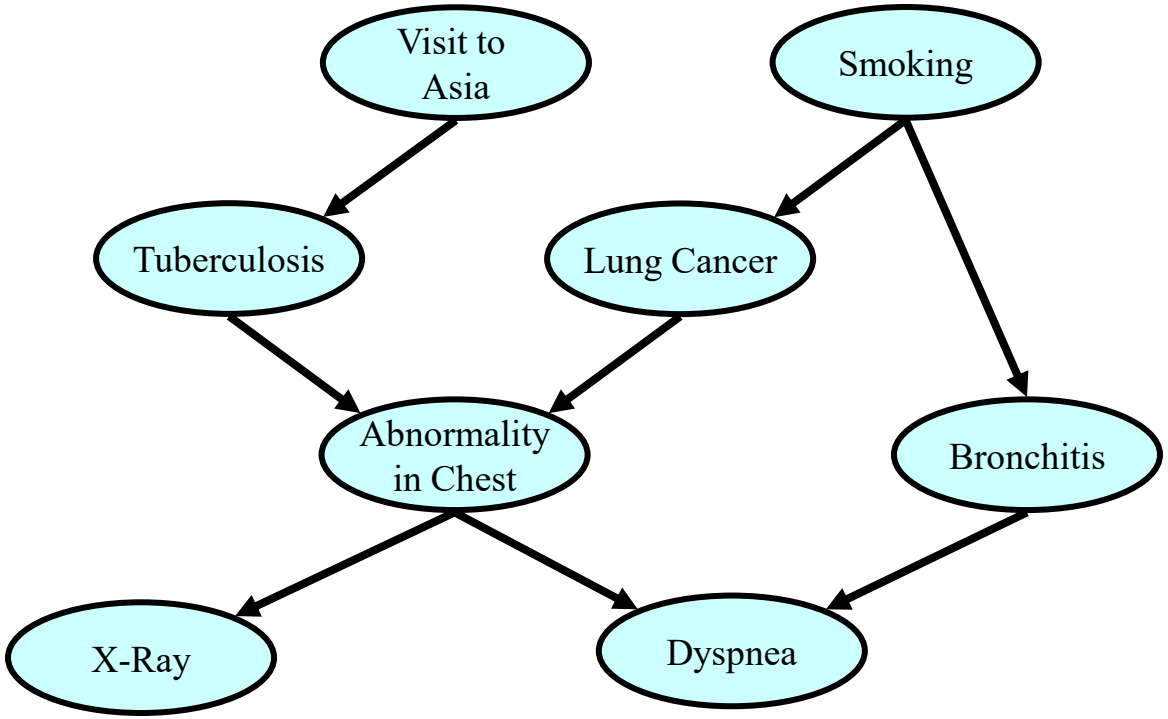
Student BN



- **If there are more than one trail between two nodes, one node influences another if there is at least one trail which allows influence to flow**

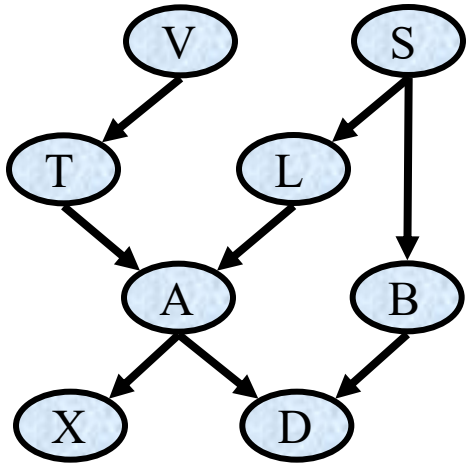
Annex 4: Example of VE algorithm

- The “Asia” network:



We are interested in *P(d)*

- Need to eliminate: *v,s,x,t,l,a,b*



Initial factors:

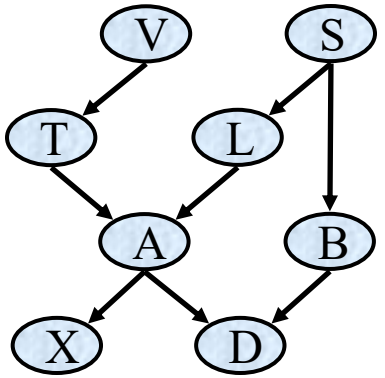
$$P(v)\,P(s)\,P(t\mid v)\,P(l\mid s)\,P(b\mid s)\,P(a\mid t,l)\,P(x\mid a)\,P(d\mid a,b)$$

Brute force:

$$P(d)=\sum_{v}\sum_{s}\sum_{x}\sum_{t}\sum_{l}\sum_{a}\sum_{b}P(v)\,P(s)\,P(t\mid v)\,P(l\mid s)\,P(b\mid s)\,P(a\mid t,l)\,P(x\mid a)\,P(d\mid a,b)$$

Eliminate variables in order:

$v \rightarrow s \rightarrow x \rightarrow t \rightarrow l \rightarrow a \rightarrow b$



Initial factors:

$$P(v)\,P(s)\,P(t \mid v)\,P(l \mid s)\,P(b \mid s)\,P(a \mid t,l)\,P(x \mid a)\,P(d \mid a,b)$$

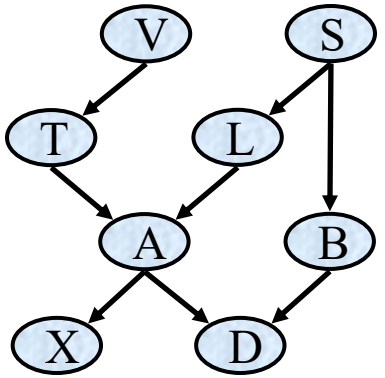
$$f_v(t) = \sum_v P(v)P(t \mid v)$$

[*We note with* $\boldsymbol{f}$ *the factor* $\boldsymbol{\tau}$]

$$\Rightarrow f_v(t)P(s)P(l \mid s)P(b \mid s)P(a \mid t,l)P(x \mid a)P(d \mid a,b)$$

Eliminate variables in order:

$$v \rightarrow s \rightarrow x \rightarrow t \rightarrow l \rightarrow a \rightarrow b$$



Initial factors:

$$P(v)\,P(s)\,P(t\,|\,v)\,P(l\,|\,s)\,P(b\,|\,s)\,P(a\,|\,t,l)\,P(x\,|\,a)\,P(d\,|\,a,b)$$

$$\Rightarrow f_v(t)\,P(s)\,P(l\,|\,s)\,P(b\,|\,s)\,P(a\,|\,t,l)\,P(x\,|\,a)\,P(d\,|\,a,b)$$

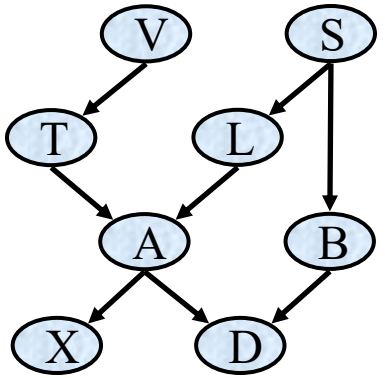
$$f_s(b,l) = \sum_s P(s)P(b\,|\,s)P(l\,|\,s)$$

[*The result of elimination may be a function of several variables*]

$$\Rightarrow f_v(t)\,f_s(b,l)\,P(a\,|\,t,l)\,P(x\,|\,a)\,P(d\,|\,a,b)$$

Eliminate variables in order:

$$v \rightarrow s \rightarrow x \rightarrow t \rightarrow l \rightarrow a \rightarrow b$$



Initial factors:

$$P(v)\,P(s)\,P(t \mid v)\,P(l \mid s)\,P(b \mid s)\,P(a \mid t,l)\,P(x \mid a)\,P(d \mid a,b)$$

$$\Rightarrow f_v(t)\,P(s)\,P(l \mid s)\,P(b \mid s)\,P(a \mid t,l)\,P(x \mid a)\,P(d \mid a,b)$$

$$\Rightarrow f_v(t)\,f_s(b,l)\,P(a \mid t,l)\,P(x \mid a)\,P(d \mid a,b)$$

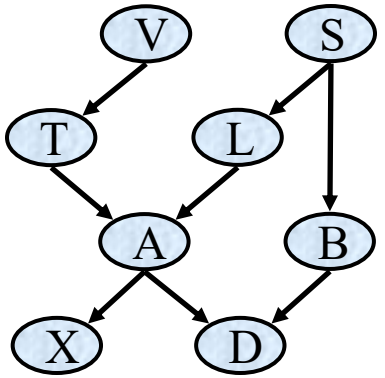
$$f_x(a) = \sum_x P(x \mid a)$$

[Note: $f_x(a) = 1$ for all values of a]

$$\Rightarrow f_v(t)\,f_s(b,l)\,f_x(a)\,P(a \mid t,l)\,P(d \mid a,b)$$

Eliminate variables in order:

$$v \rightarrow s \rightarrow x \rightarrow t \rightarrow l \rightarrow a \rightarrow b$$



Initial factors:

$$P(v)\,P(s)\,P(t \mid v)\,P(l \mid s)\,P(b \mid s)\,P(a \mid t,l)\,P(x \mid a)\,P(d \mid a,b)$$

$$\Rightarrow f_v(t)\,P(s)\,P(l \mid s)\,P(b \mid s)\,P(a \mid t,l)\,P(x \mid a)\,P(d \mid a,b)$$

$$\Rightarrow f_v(t)\,f_s(b,l)\,P(a \mid t,l)\,P(x \mid a)\,P(d \mid a,b)$$

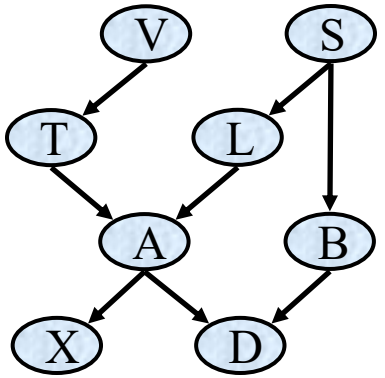
$$\Rightarrow f_v(t)\,f_s(b,l)\,f_x(a)\,P(a \mid t,l)\,P(d \mid a,b)$$

$$f_t(a,l) = \sum_t f_v(t) P(a \mid t,l)$$

$$\Rightarrow f_s(b,l) f_x(a) f_t(a,l) P(d \mid a,b)$$

Eliminate variables in order:

$$v \rightarrow s \rightarrow x \rightarrow t \rightarrow l \rightarrow a \rightarrow b$$



Initial factors:

$$P(v)\,P(s)\,P(t \mid v)\,P(l \mid s)\,P(b \mid s)\,P(a \mid t,l)\,P(x \mid a)\,P(d \mid a,b)$$

$$\Rightarrow f_v(t)\,P(s)\,P(l \mid s)\,P(b \mid s)\,P(a \mid t,l)\,P(x \mid a)\,P(d \mid a,b)$$

$$\Rightarrow f_v(t)\,f_s(b,l)\,P(a \mid t,l)\,P(x \mid a)\,P(d \mid a,b)$$

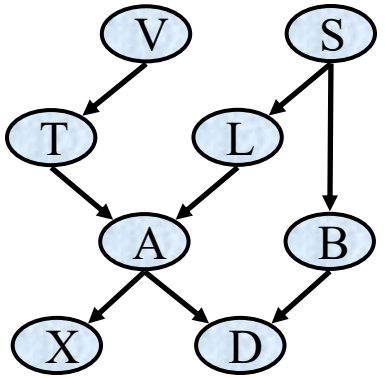
$$\Rightarrow f_v(t)\,f_s(b,l)\,f_x(a)\,P(a \mid t,l)\,P(d \mid a,b)$$

$$\Rightarrow f_s(b,l)\,f_x(a)\,f_t(a,l)\,P(d \mid a,b)$$

$$f_l(a,b) = \sum_l f_s(b,l) f_t(a,l) \quad \Rightarrow \quad f_l(a,b) f_x(a) P(d \mid a,b)$$

Eliminate variables in order:

$$v \rightarrow s \rightarrow x \rightarrow t \rightarrow l \rightarrow a \rightarrow b$$



Initial factors:

$$P(v)\,P(s)\,P(t\mid v)\,P(l\mid s)\,P(b\mid s)\,P(a\mid t,l)\,P(x\mid a)\,P(d\mid a,b)$$

$$\Rightarrow f_v(t)\,P(s)\,P(l\mid s)\,P(b\mid s)\,P(a\mid t,l)\,P(x\mid a)\,P(d\mid a,b)$$

$$\Rightarrow f_v(t)\,f_s(b,l)\,P(a\mid t,l)\,P(x\mid a)\,P(d\mid a,b)$$

$$\Rightarrow f_v(t)\,f_s(b,l)\,f_x(a)\,P(a\mid t,l)\,P(d\mid a,b)$$

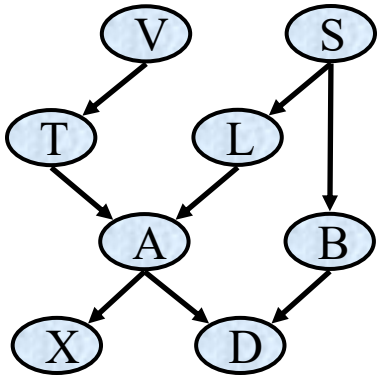
$$\Rightarrow f_s(b,l)\,f_x(a)\,f_t(a,l)\,P(d\mid a,b)$$

$$\Rightarrow \boxed{f_l(a,b)}\,\boxed{f_x(a)}\,\boxed{P(d\mid a,b)}$$

$$f_a(b,d)=\sum_a f_l(a,b)f_x(a)p(d\mid a,b) \Rightarrow f_a(b,d)$$

Eliminate variables in order:

$$v \rightarrow s \rightarrow x \rightarrow t \rightarrow l \rightarrow a \rightarrow b$$



Initial factors:

$$P(v)\,P(s)\,P(t\mid v)\,P(l\mid s)\,P(b\mid s)\,P(a\mid t,l)\,P(x\mid a)\,P(d\mid a,b)$$

$$\Rightarrow f_v(t)\,P(s)\,P(l\mid s)\,P(b\mid s)\,P(a\mid t,l)\,P(x\mid a)\,P(d\mid a,b)$$

$$\Rightarrow f_v(t)\,f_s(b,l)\,P(a\mid t,l)\,P(x\mid a)\,P(d\mid a,b)$$

$$\Rightarrow f_v(t)\,f_s(b,l)\,f_x(a)\,P(a\mid t,l)\,P(d\mid a,b)$$

$$\Rightarrow f_s(b,l)\,f_x(a)\,f_t(a,l)\,P(d\mid a,b)$$

$$\Rightarrow f_l(a,b)\,f_x(a)\,P(d\mid a,b)$$

$$\Rightarrow \boxed{f_a(b,d)} \longrightarrow f_b(d)=\sum_b f_a(b,d) \Rightarrow f_b(d)$$

Intermediate factors

In our previous example:

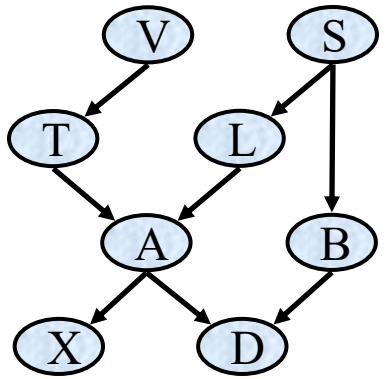
$$v \to s \to x \to t \to l \to a \to b$$

$f_v(t)$
$f_s(b,l)$
$f_x(a)$
$f_t(a,l)$
$f_l(a,b)$
$f_a(b,d)$
$f_b(d)$

With a different ordering:

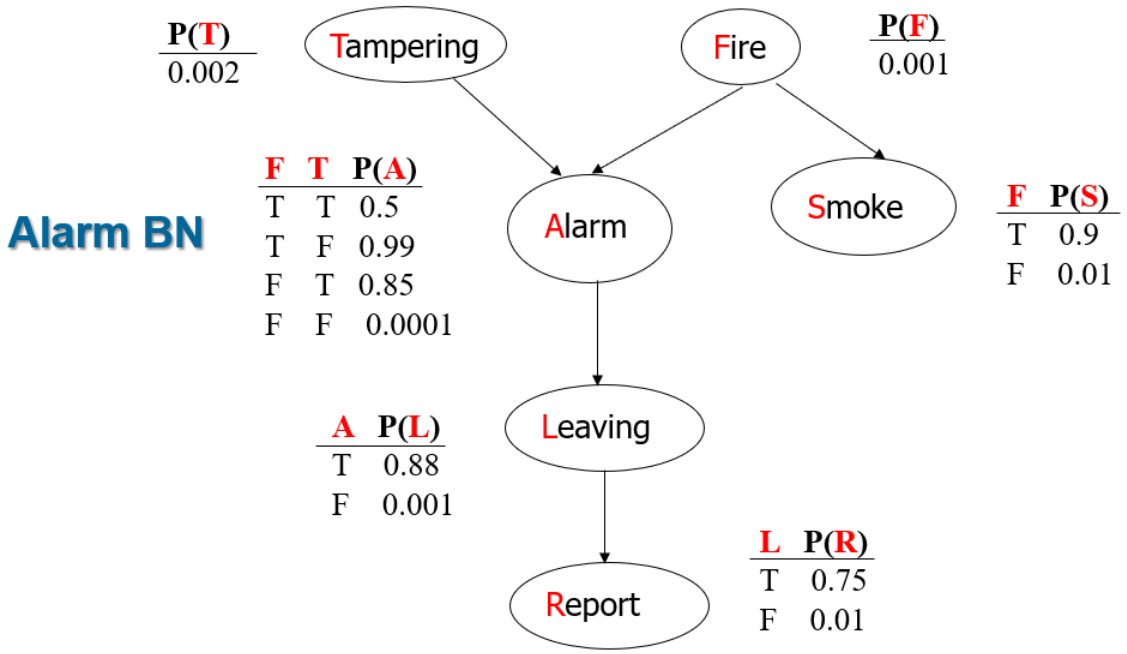
$$a \to b \to x \to t \to v \to s \to l$$

$g_a(l,t,d,b,x)$
$g_b(l,t,d,x,s)$
$g_x(l,t,d,s)$
$g_t(l,t,s,v)$
$g_v(l,d,s)$
$g_s(l,d)$
$g_l(d)$



Complexity is exponential in the size of these factors!

Annex 5: Exercise



Show that:

- If the prior probabilities (with no evidence) of each variable are:

  P(Tampering) = 0.02

  P(Fire) = 0.1

  P(Report) = 0.028

  P(Smoke) = 0.0189

- Suppose that **Report** was observed, then

  P(Tampering|Report) = 0.399

  P(Fire|Report) = 0.2305

  P(Smoke|Report) = 0.215

- The probability of both Tampering and Fire are increased by the Report
- Because Fire is increased, so is the probability of Smoke

Annex 6: How to build the Junction tree

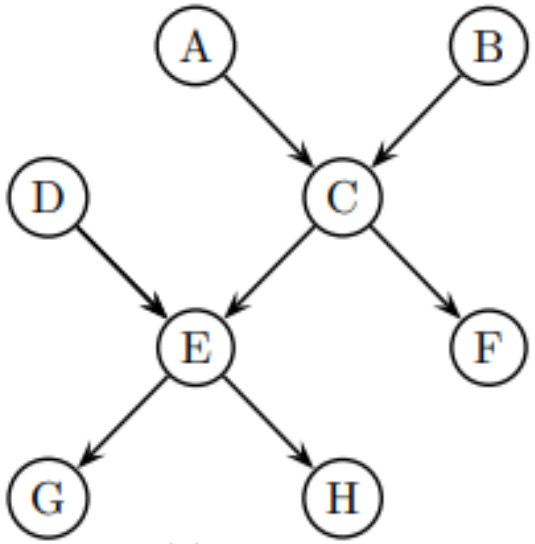
Moralize the graph

- Take the Bayesian Network (DAG)
- **Connect all parents** of each node (i.e., “marry the parents”)

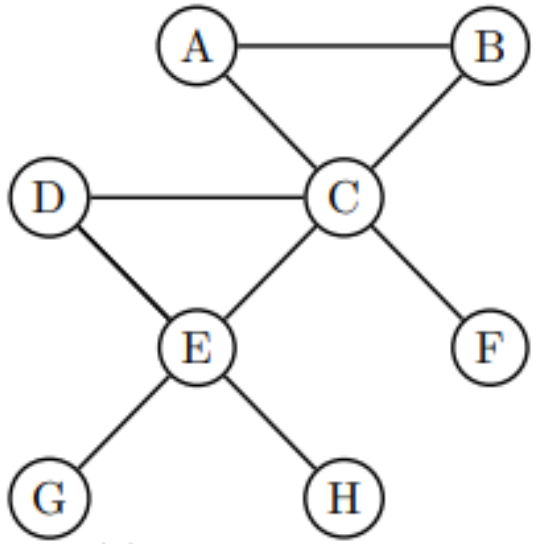
  Example: If both A and B are parents of C, add an edge between A and B
- **Drop all directions** on edges
- You now have a **moral graph** (undirected)

(*Definition: the **moral graph** of a directed acyclic graph is formed by adding edges between all pairs of non-adjacent nodes that have a common child, and then making all edges in the graph undirected*)

Moral graph example



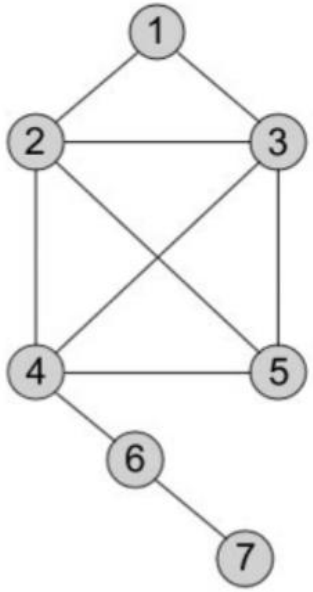
(a) DAG



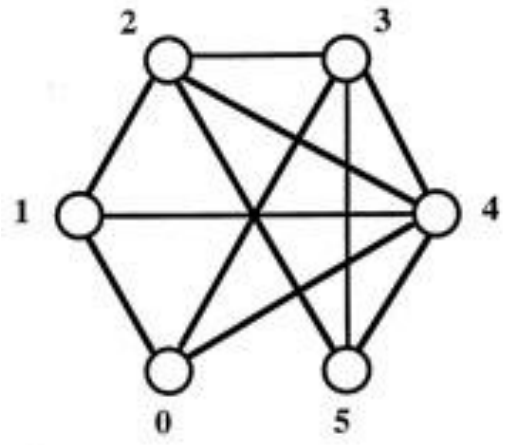
(b) Moral Graph

Clique and maximal clique

- A **clique C** of a graph is a subset of vertices of un undirected graph $G = (V, E)$ such that every two distinct vertices in the clique are adjacents (= the induced subgraph is complete)
- A **maximal clique** is a clique that cannot be extended by including one more adjacent vertex (a clique that is not a subset of a larger clique)



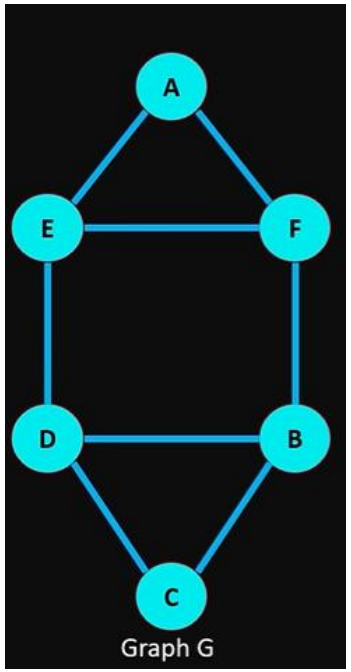
1-2-3	maximal clique
2-3-4	clique
3-5-2	clique
5-4-3	clique
4-2-5	clique
2-3-4-5	maximum clique
4-6-7	not clique



The maximum clique size is 4 and the maximum clique contains the nodes 2,3,4,5

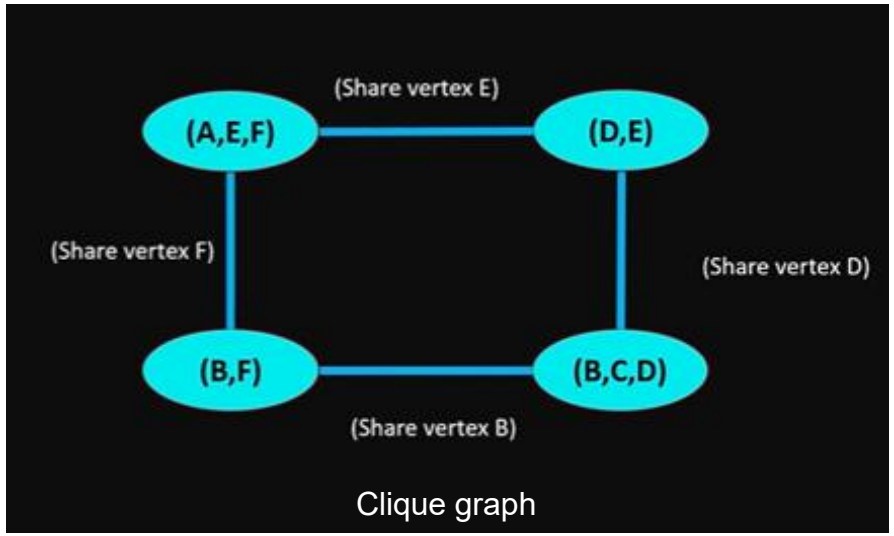
Clique graph

- A **clique graph K(G)** of an undirected graph G is a graph with a vertex for each maximal clique in G
- Two vertices are adjacent in K(G) when their corresponding maximal cliques share at least one vertex



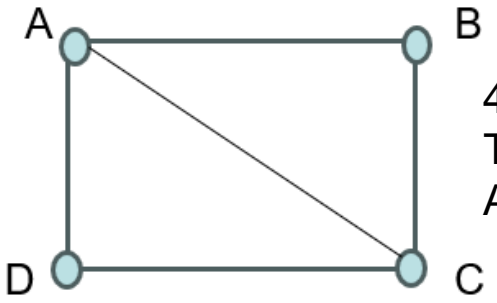
Maximal cliques:

- (A, E, F)
- (D, E)
- (B, F)
- (B, C, D)



Triangulate the graph

- A **junction tree** requires to be built on a **chordal graph**
- To build a junction tree from a clique graph, we need to convert the graph into a chordal graph
- A **chord** is an **edge connecting two non-adjacent nodes in some cycle**
- **Chordal Graph -** A graph where **every cycle of four or more vertices has a chord** (an edge that is not part of the cycle but connects two vertices of the cycle)



4 cycle: A, B, C ,D
The graph is chordal
AC is the chord in the cycle



4 cycle: A, B, C ,D
The graph is no chordal

Triangulate the graph

- If the graph is not already chordal then it must **be made so by triangulation (triangulate the graph)**
- Triangulation adds edges to remove cycles of length 4 or more without a chord

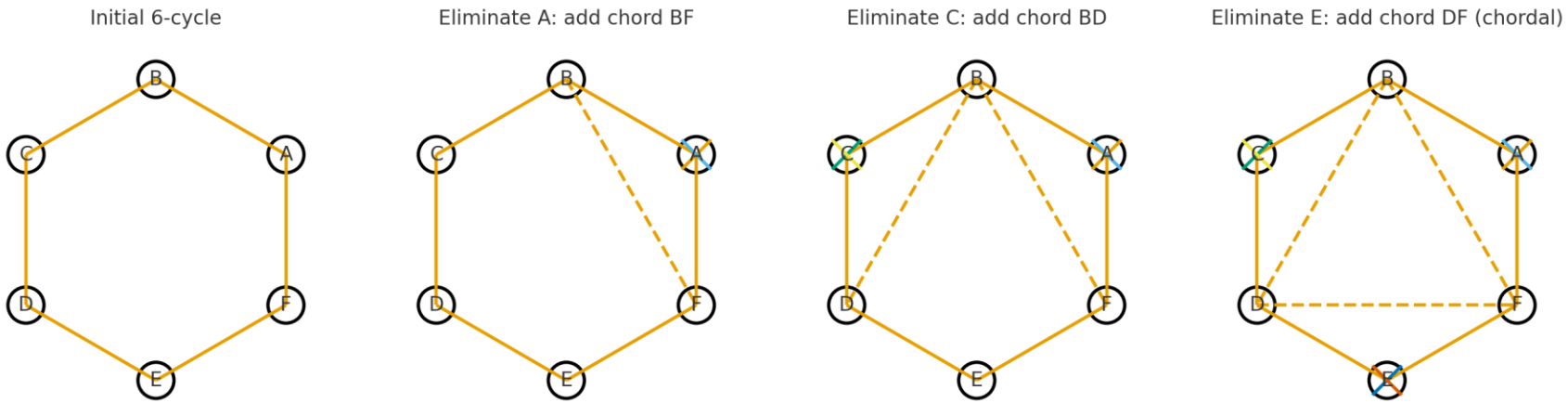
1. Choose an elimination order of nodes – make a list
2. For each node in the order: connect any two unconnected neighbors of the node, creating a clique among them
3. Remove the node from the list
4. Repeat until all nodes are removed. The result is a **chordal (triangulated) graph**

For elimination, we can use a **heuristic, e.g., Minimum degree heuristic: c**hose first vertices with the smallest degree (the fewest neighbors) for elimination, assuming that fewer neighbors will generally lead to fewer edges

Triangulate the graph example

Elimination order $A \to C \to E \to B \to D \to F$

- Eliminate A: connect neighbors $B$and $F \to$ add chord BF
- Eliminate C: connect neighbors $B$and $D \to$ add chord BD
- Eliminate E: connect neighbors $D$and $F \to$ add chord DF
- Remaining eliminations add nothing; the graph is now chordal



Why chordal?

Justifying why a junction tree requires to be built on a **chordal graph**

- **The running intersection says** – for each pair of clusters U, V, all nodes on the path between U and V contain $U \cap V$
- This property is crucial because it ensures that all information about any variable is shared consistently across the tree, making it possible to perform exact inference without duplicating or missing information
- Chordal graphs make this possible because they allow to organize the graph's maximal cliques into a tree structure (junction tree) where the running intersection property holds
- In non-chordal graphs, it's impossible to create a junction tree that satisfies this property without adding extra edges, leading to cycles that violate the property

Potential of a clique

Definition:
A **clique potential** $\phi_C(C)$ is a non-negative function defined over the variables in a **clique** C of a graph = a product of the factors in the clique
It represents the *local contribution* of that clique to the overall JPD

$$\phi_C(X_C) \geq 0, \quad \forall X_C$$

$$P(X_1, X_2, \ldots, X_n) = \frac{1}{Z} \prod_{C \in \text{Cliques}} \phi_C(X_C)$$

A **potential** captures the **local interaction** between variables inside the clique

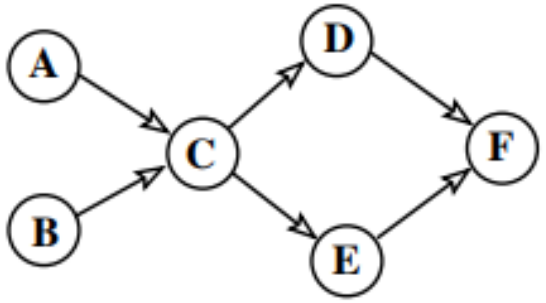
- tells you how "compatible" a given combination of those variables' values is
  - If a value combination fits the model well then $\phi_C(C)$ is **large**
  - If it's unlikely then $\phi_C(C)$ is **small** (possibly 0)
- It does **not** need to be a probability yet — only a *nonnegative score*
- Probabilities are obtained **after normalization**

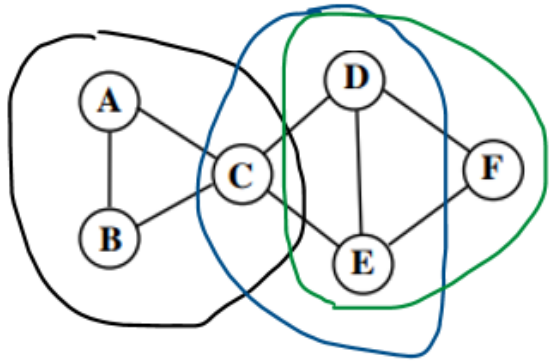
$$P(X_C) = \frac{\phi_C(X_C)}{\sum_{X_C} \phi_C(X_C)}$$

Potential of a clique - example

$\phi_1(A,B,C) = P(A)P(B)P(C|A,B)$
$\phi_2(C,D,E) = P(D|C)P(E|C)$
$\phi_3(D,E,F) = P(F|D,E)$

$P(A,B,C,D,E,F) = \phi_1(A,B,C)\ \phi_2(C,D,E)\ \phi_3(D,E,F)$

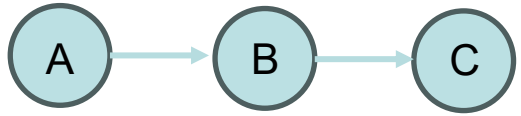




Annex 7: Belief propoagation

A simple example of belief propagation

Joint probability P(A,B,C)=P(A)P(B|A)P(C|B)



- **Build junction tree**

  C1 = {A,B} C2 = {B,C}

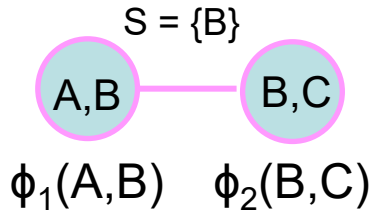
  {A,B} — {B,C}

- Sepset S = {B}
- **Assign potentials**

  $\phi_1(A,B) = P(A)P(B|A) = P(A,B)$

  $\phi_2(B,C) = P(C|B)$

Ensures the **family preservation property** of the junction tree:
every variable and its parents appear together in at least one clique



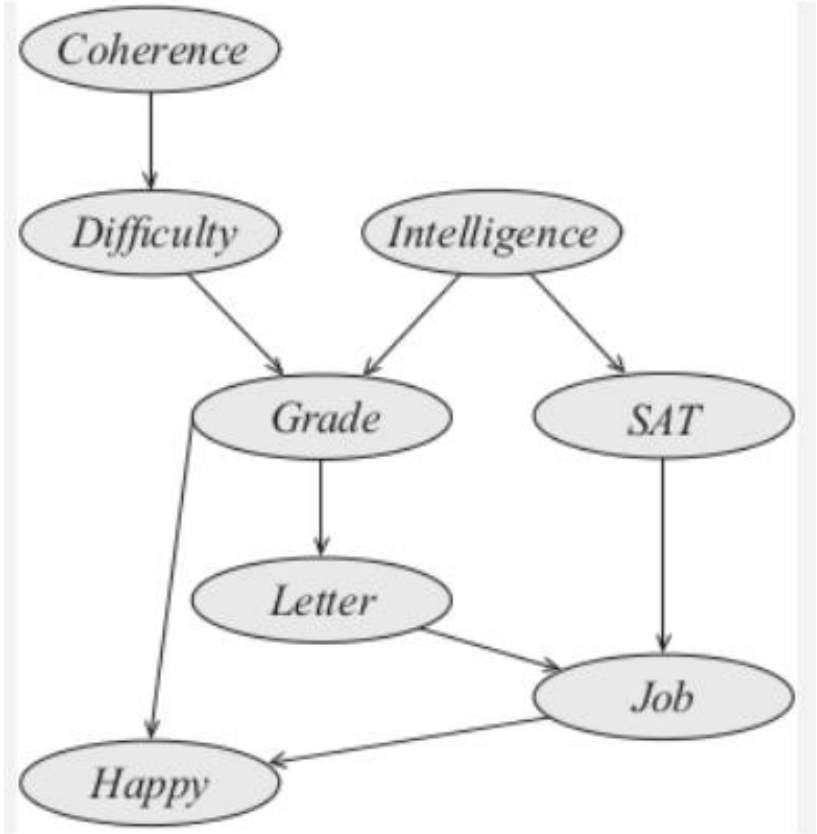
- Message from {A,B} → {B,C} $m_{1\to 2}(B) = \sum_A \phi_1(A,B)$
- Combine at {B,C} $\beta_2(B,C) = \phi_2(B,C) \cdot m_{1\to 2}(B)$
- Marginalize if needed $P(C) = \sum_B \beta_2(B,C)$

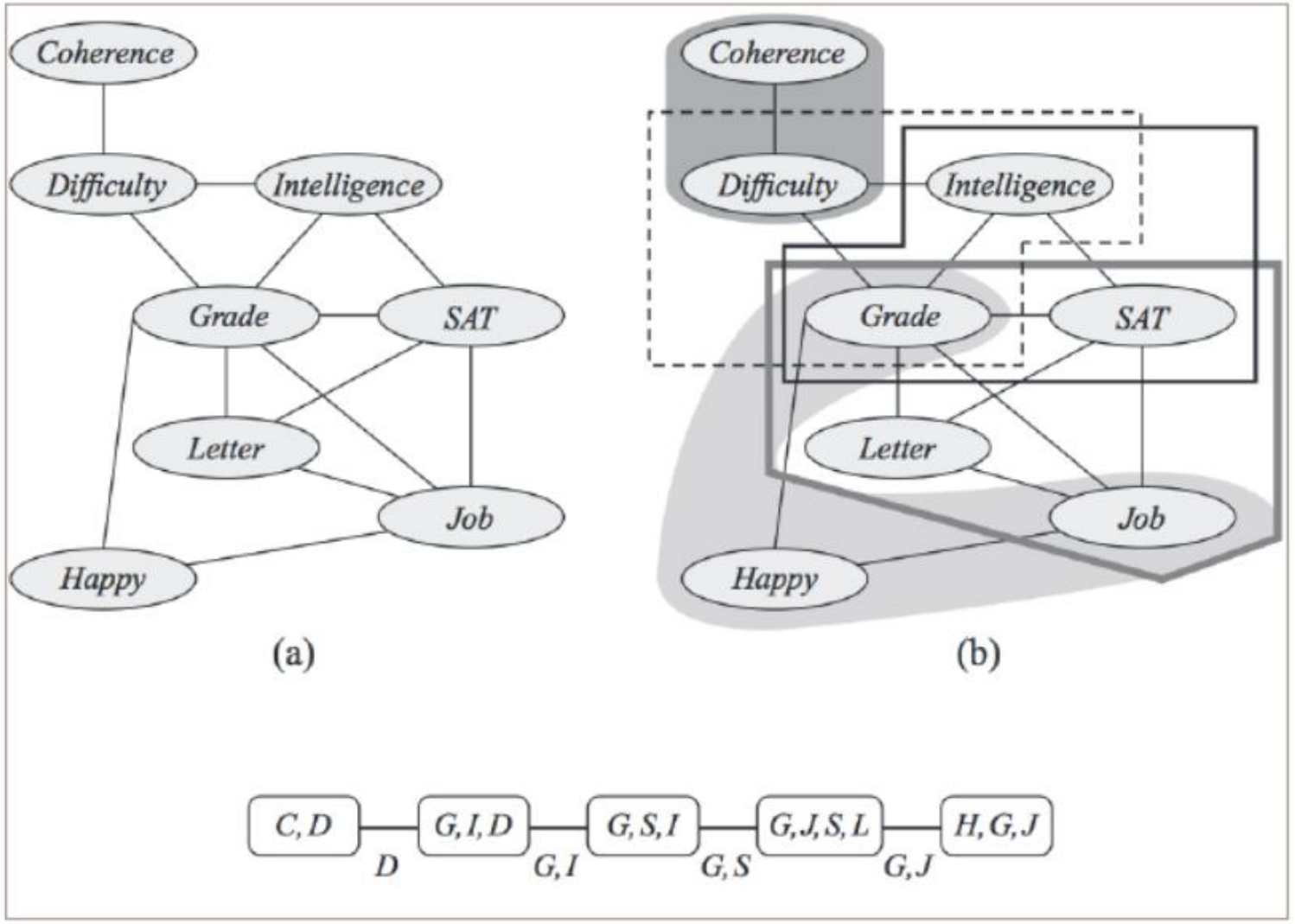
This gives an exact marginal probability P(C), the same result as **Variable Elimination**, but computed through **local message passing**

Example of JT algorithm

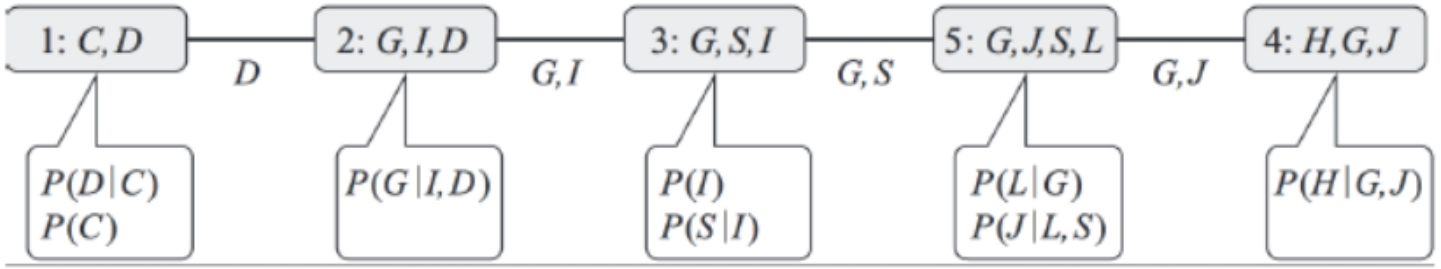
Credit Sargur Srihari

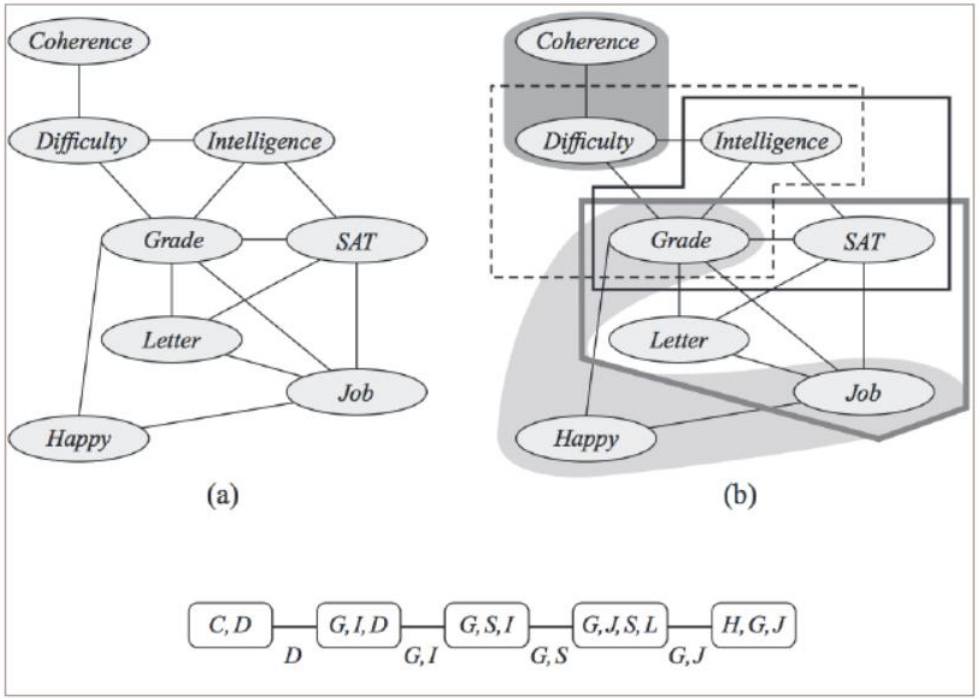
- A Student BN





- This tree satisfies Running Intersection Property and the Family Preservation property





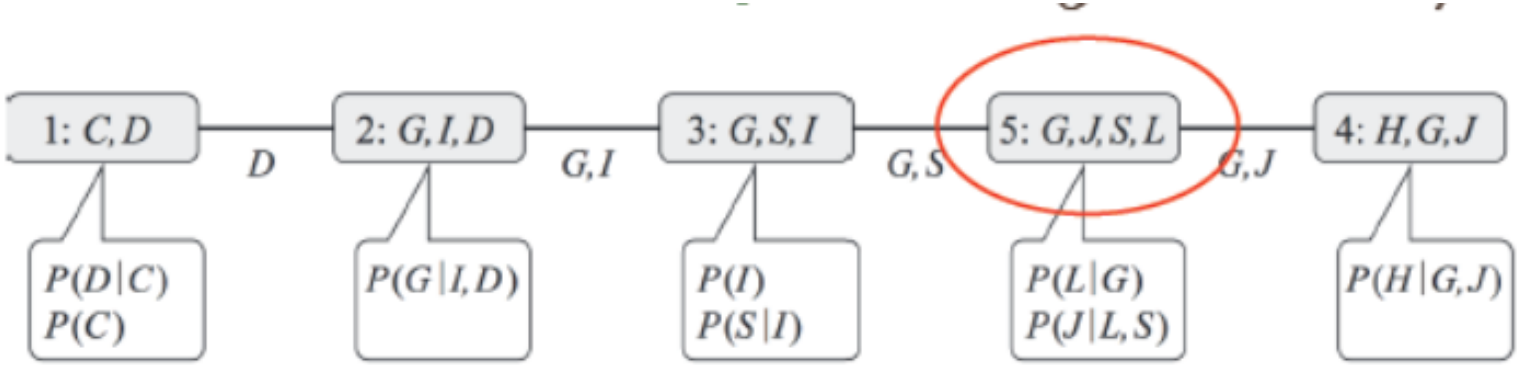
Determine *P(J)*

Assign α: initial factors (CPDs) to cliques

• First step: Generate initial set of potentials by multiplying out the factors

– E.g., $\psi_5(J,L,G,S) = f_L(L,G) * f_J(J,L,S)$ – *we denote here by* $\boldsymbol{\psi}$ *the potential*

• Root is selected to have variable *J*, since we are interested in determining *P(J),* e.g., *C5*

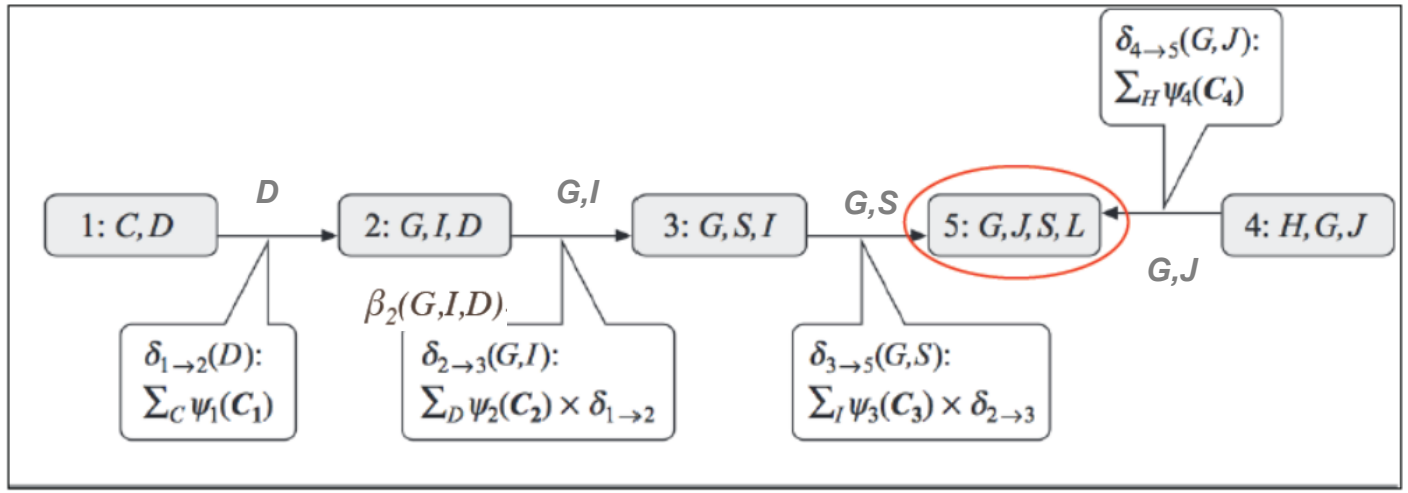


$$m_{i\to j}(S_{ij}) = \sum_{x_c \setminus S_{ij}} \psi_c(x_c) \prod_{\ell \in N(i)\setminus j} m_{\ell\to i}(S_{\ell i}).$$

$$\beta_c(x_c) = \psi_c(x_c) \prod_{\ell \in N(i)} m_{\ell\to i}(S_{\ell i}).$$

Root=C_5

To compute $P(J)$

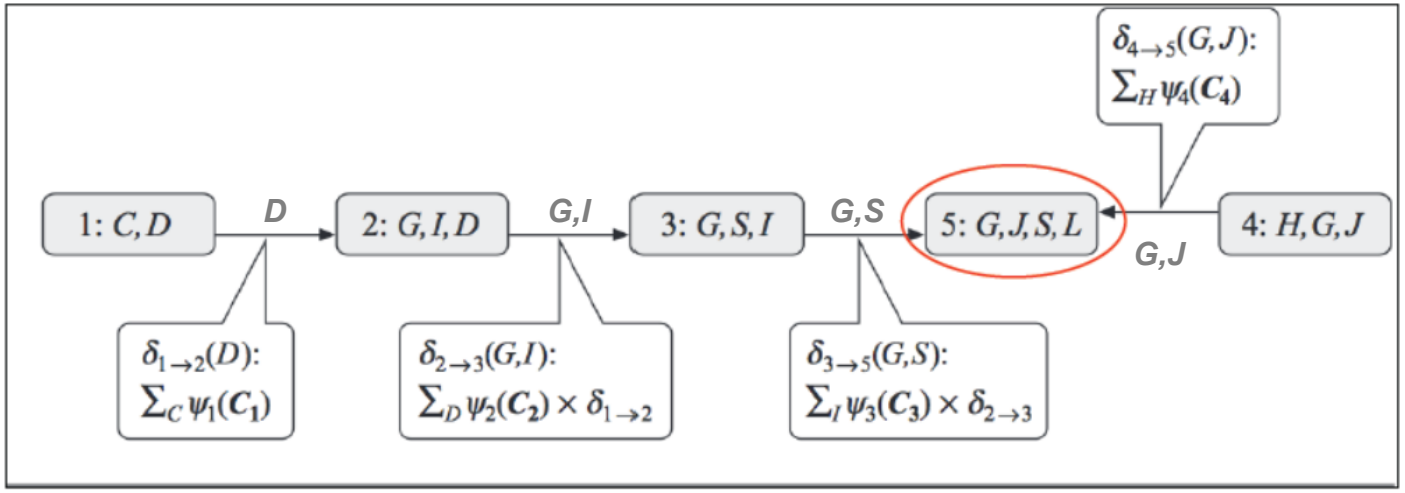


In C_1: eliminate C by performing $\sum_C \psi_1(C,D)$
The resulting factor has scope D. We send it as a message $\delta_{1\text{-}2}(D)$ to C_2

In C_2: We define $\beta_2(G,I,D)=\delta_{1\text{-}2}(D)\psi_2(G,I,D)$. We then eliminate D to get a factor over G,I. The resulting factor is $\delta_{2\text{-}3}(G,I)$ which is sent to C_3.

Root=C_5

To compute $P(J)$



$$\delta_{1\to2}(D) = \sum_C \psi_1(C,D)$$

$$\delta_{2\to3}(G,I) = \sum_D \psi_2(G,I,D)\ \delta_{1\to2}(D)$$

$$\delta_{3\to5}(G,S) = \sum_I \psi_3(G,S,I)\ \delta_{2\to3}(G,I)$$

$$\delta_{4\to5}(G,J) = \sum_H \psi_4(H,G,J)$$

$$\beta_5(G,J,S,L) = \psi_5(G,J,S,L)\ \delta_{3\to5}(G,S)\ \delta_{4\to5}(G,J)$$

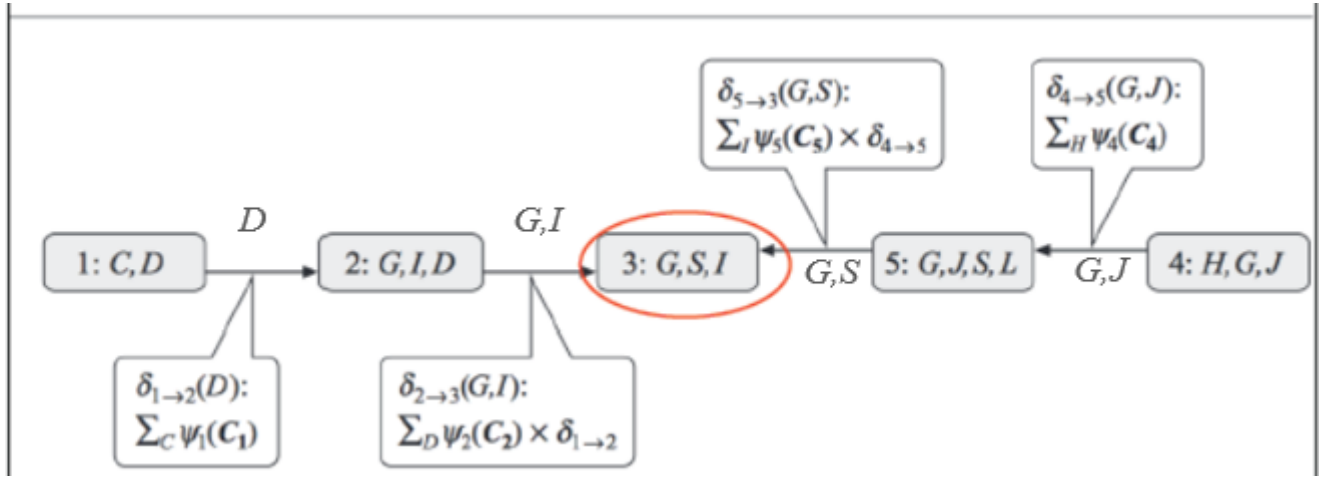
$$\tilde{P}(J) = \sum_{G,S,L} \beta_5(G,J,S,L)$$

$$P(J) = \frac{\tilde{P}(J)}{\sum_j \tilde{P}(j)}$$

Root=C_3

To compute $P(G)$

$$\tilde{p}(x) \propto \sum_{x_c \setminus x} \beta_c(x_c).$$



$$\delta_{1\to2}(D) = \sum_C \psi_1(C,D), \quad \delta_{2\to3}(G,I) = \sum_D \psi_2(G,I,D)\ \delta_{1\to2}(D)$$

$$\delta_{4\to5}(G,J) = \sum_H \psi_4(H,G,J), \qquad \delta_{5\to3}(G,S) = \sum_{J,L} \psi_5(G,J,S,L)\ \delta_{4\to5}(G,J)$$

$$\beta_3(G,S,I) = \psi_3(G,S,I)\ \delta_{2\to3}(G,I)\ \delta_{5\to3}(G,S)$$

$$\tilde{P}(G) = \sum_{S,I} \beta_3(G,S,I), \qquad P(G) = \frac{\tilde{P}(G)}{\sum_g \tilde{P}(g)}$$